Modeling Chemical Kinetics with ChemKANs

Hasan Berke Bankoglu Student ID: 12432802
MSc Data Science, TU Wien
Interdisciplinary Project in Data Science (194.147)

Main Supervisor: Ass. Prof. DI Dr. Markus Bösenhofer
Co-Supervisor: Univ.-Prof. Dr. Thomas Gärtner

Abstract

This project investigates the reproducibility and transferability of Chemistry Kolmogorov-Arnold Networks (ChemKANs) for learning chemical kinetics. An independent PyTorch implementation was developed to reproduce the original biodiesel-production and hydrogen-air-combustion benchmarks and extend the architecture to methane-air oxidation. The workflow followed CRISP-DM adapted to scientific machine learning, emphasizing controlled experiments, numerical validation, and reproducibility. The principal species trajectories and, for hydrogen, ignition behavior were reproduced after incorporating methodological details clarified by the authors and documented in their released biodiesel implementation. These details improved agreement with the published results, although some quantitative differences remained [2, 4]. The methane extension reproduced the principal species and ignition dynamics with a held-out time-averaged normalized MSE of 1.16×10^{-3}. Computational benchmarks demonstrated that inference efficiency depends strongly on implementation: the PyTorch-trained ChemKAN was slower than Cantera when evaluated in PyTorch, whereas a Julia inference port achieved approximately $2.5\times$ and $4.0\times$ speed-ups over Cantera for hydrogen and methane, respectively, under the evaluated configurations. Analysis of the learned KAN functions and instantaneous species production rates revealed discrepancies in the learned kinetics that were not apparent from agreement with integrated reference trajectories alone.

Contents

1 Motivation, Problem Statement and Research Questions **3**
1.1 Motivation and problem statement . . . 3
1.2 Research questions . . . 3
1.3 Project scope and adaptations . . . 3

2 Methodology and Process **3**
2.1 CRISP-DM adapted to scientific machine learning . . . 4
2.2 Chemical kinetics and ChemKAN formulation . . . 4
2.3 Data generation and preparation . . . 6
2.3.1 Biodiesel production . . . 6
2.3.2 Hydrogen–air combustion . . . 7
2.3.3 Methane-air oxidation extension . . . 7
2.4 Experiment design and reproducibility controls . . . 8
2.5 Forward Sensitivity Analysis . . . 8
2.6 Evaluation metrics and baselines . . . 9

3 Results and Evaluation **10**
3.1 Biodiesel reproduction . . . 10
3.2 Hydrogen-air combustion reproduction . . . 13
3.3 FSA validation and sensitivity-backend findings . . . 17

3.4 Methane extension . . . 18
3.5 Computational efficiency . . . 19
3.6 Interpretability . . . 20
3.7 Evaluation against proposal success criteria . . . 22

4 Discussion and State-of-the-Art Comparison **23**
4.1 RQ1: Reproducibility of ChemKAN . . . 23
4.2 RQ2: Transfer to methane-air oxidation . . . 23
4.3 RQ3: Comparison with neural and numerical references . . . 23
4.4 Reproducibility lessons . . . 24
4.5 Limitations . . . 25

5 Conclusion **25**

6 Domain-Specific Lecture **26**

A Implementation and Training Settings **28**

B Source and Reproducibility Notes **28**

C Training Histories **28**

D Additional Interpretability Plots **30**

1 Motivation, Problem Statement and Research Questions

1.1 Motivation and problem statement

Detailed chemical kinetics are typically represented by coupled nonlinear ordinary differential equations (ODEs) with widely separated time scales. In combustion, evaluating detailed chemical source terms and integrating stiff dynamics can become a major computational bottleneck. Machine-learning surrogate models therefore aim to approximate either the evolving thermochemical state or the chemical source terms while retaining sufficient physical fidelity and generalization [8, 12].

ChemKAN combines the neural-ODE viewpoint with Kolmogorov-Arnold Networks (KANs), using a chemistry-specific architecture that separates kinetic and thermodynamic information flow [8]. KANs replace fixed scalar connection weights with learnable univariate functions, providing a different inductive bias from conventional multi-layer perceptrons (MLPs) [10]. The original ChemKAN study reports parameter-efficient models, robustness to noisy data, and a compact 344-parameter hydrogen surrogate that reproduces ignition behavior while offering computational acceleration [8]. The central problem of this project is whether these findings can be reproduced in an independent PyTorch implementation and whether the architecture transfers to a larger chemical system.

1.2 Research questions

The research questions remain aligned with the approved proposal:

RQ1: How accurately can a PyTorch reimplementation of ChemKAN [8] reproduce the predictive accuracy reported in [8] for the hydrogen combustion and biodiesel production benchmarks?

RQ2: How well does ChemKAN perform in terms of predictive accuracy (test MSE) and computational efficiency (inference time relative to a traditional numerical solver) when applied to a chemical system not covered in [8], such as methane oxidation?

RQ3: How does ChemKAN compare to (a) the neural baselines used in [8] in terms of generalization to unseen initial conditions and robustness to noisy training data, and (b) traditional ODE solvers in terms of computational efficiency?

1.3 Project scope and adaptations

The project remained a reproduction and extension study of ChemKAN, but the reproduction protocol evolved as additional evidence became available. The first implementation was based on the original ChemKAN study and the related KAN, LeanKAN, and KAN-ODE literature [8, 10, 9, 7]. Considerable diagnostic work and controlled ablations were carried out after this initial implementation because important quantitative discrepancies remained: the species dynamics could often be captured, but the hydrogen thermodynamic stage did not reproduce ignition reliably and the loss levels remained substantially higher than the reported results.

Written clarifications from the authors were later used to resolve methodological details that could not be determined reliably from the paper alone [2]. A released biodiesel Julia example also became available and was used as an additional source for implementation-level comparison [4]. These sources are not treated as if they had been specified in the original paper. The report therefore distinguishes among (i) the printed paper, (ii) author clarification and released source code, and (iii) implementation choices made in this project. Earlier paper-only results are retained in the development(`dev`) branch to document how the reproduction evolved as additional methodological details became available. The release repository contains the selected final runs, their checkpoints and evaluation artifacts; it does not include every exploratory run.

2 Methodology and Process

The project is designed as a **reproducibility and extension** study of ChemKAN [8]. The methodological workflow follows **CRISP-DM** [1], adapted from an industrial data-mining workflow to scientific machine learning. In addition, the project applies controlled experiment design, explicit provenance tracking, numerical-method validation, train/test separation, and reproducible computational practice.

2.1 CRISP-DM adapted to scientific machine learning

Table 1: CRISP-DM phases and their adaptation in this project.

CRISP-DM phase	Application in this project
Problem understanding	Study the ChemKAN paper and related KAN, LeanKAN, KAN-ODE, Neural ODE, and chemical-kinetics literature; review homogeneous-reactor kinetics, Arrhenius rates, stiff ODEs, and numerical integration; formalize the architecture, training procedure, benchmark claims, and reproduction targets; record ambiguous or under-specified methodological details; define the reproducibility, baseline-comparison, and transfer questions.
Data understanding	Inspect biodiesel and combustion trajectories, stiffness, state ranges, ignition behavior, initial-condition coverage, and noise construction.
Data preparation	Generate/reference trajectories; define train/test initial conditions; apply normalization; construct controlled noise levels; generate dense temperature caches for staged training.
Modeling	Implement ChemKAN, DeepONet reference models, direct-autograd and FSA backends, staged hydrogen training, and the methane extension.
Evaluation	Use trajectory MSE, clean/noisy test error, dense-grid generalization, ignition-delay error, integration failures, parameter counts, and runtime.
Deployment / communication	Maintain a version-controlled repository with configurations, checkpoints, hashes, experiment logs, reproducible plotting scripts, the final report, and poster.

2.2 Chemical kinetics and ChemKAN formulation

For a homogeneous reactor, the thermochemical state is written as

$$\mathbf{u}(t) = [Y_1, \ldots, Y_m, T](t), \tag{1}$$

where Y_i is the mass fraction of species i, T is temperature, ρ is density, W_i is molecular weight, and $\dot{\omega}_i$ is the molar production or consumption rate. The chemical and thermal dynamics are written in the ChemKAN paper as [8]

$$\frac{\mathrm{d}Y_i}{\mathrm{d}t} = \frac{1}{\rho} W_i \dot{\omega}_i, \tag{2}$$

$$\frac{\mathrm{d}T}{\mathrm{d}t} = -\sum_{i=1}^{m} \frac{h_i \dot{Y}_i}{c_p}, \tag{3}$$

$$\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = f(\mathbf{u}, t), \tag{4}$$

where h_i is the species enthalpy and c_p is the mixture-averaged specific heat. The generic form $\mathrm{d}\mathbf{u}/\mathrm{d}t = f(\mathbf{u}, t)$ is the ODE that must be integrated to obtain the trajectory.

KAN and LeanKAN layers. A KAN layer applies a learnable univariate activation $\phi_{l,\alpha,\beta}$ to each connection from input node β to output node α, rather than using only scalar weights with fixed node activation functions [10]. ChemKAN uses Gaussian radial-basis-function (RBF) activations. In the paper, an activation is written as [8]

$$\phi_{l,\alpha,\beta}(x) = \sum_{i=1}^{N} w^{\psi}_{l,\alpha,\beta,i}\, \psi(|x - c_i|) + w^{b}_{l,\alpha,\beta}\, b(x), \tag{5}$$

$$\psi(r) = \exp\left(-\frac{r^2}{2h^2}\right), \tag{6}$$

where c_i are RBF centers, h is the grid spacing/spreading parameter, and $b(x)$ is the base activation. The paper states a Swish base activation, while the authors later clarified that the final ChemKAN models used no base activation and that the base term remained from an earlier revision [2]. The released biodiesel implementation

also disables the base term [4]. Its RBF is implemented as $\exp[-((x-c)/h)^2]$; the PyTorch author-match uses the equivalent Gaussian width by setting the library width to $h/\sqrt{2}$.

LeanKAN introduces multiplication without the extra subnodes of MultKAN [9]. For layer l, the paper writes

$$\Psi_l^{\text{lean}}(\mathbf{x}_l) = \mathbf{y}_l^{\text{mult}} + \mathbf{y}_l^{\text{add}} \in \mathbb{R}^{n_{l+1}}, \tag{7}$$

$$y_{l,i}^{\text{mult}} = \prod_{j=1}^{n_l^{\text{mu}}} \phi_{l,i,j}(x_{l,j}) \qquad \text{for } i \in \{1, 2, \ldots, n_{l+1}\} \subset \mathbb{N}, \tag{8}$$

$$y_{l,i}^{\text{add}} = \sum_{j=n_l^{\text{mu}}+1}^{n_l} \phi_{l,i,j}(x_{l,j}) \qquad \text{for } i \in \{1, 2, \ldots, n_{l+1}\} \subset \mathbb{N}, \tag{9}$$

Thus n_l^{mu} controls how many input activations enter multiplicatively. When $n_l^{\text{mu}} = 0$, the multiplicative branch is disabled and the layer reduces to the ordinary additive KAN.

ChemKAN architecture. Following the notation of the ChemKAN paper, let the full state be $\mathbf{u} = [\widetilde{\mathbf{u}}, T]$, where $\widetilde{\mathbf{u}}$ contains the species states. The chemistry-specific architecture is [8]

$$\frac{\mathrm{d}\widetilde{\mathbf{u}}}{\mathrm{d}t} = \mathrm{KAN}_{\text{kin}}(\mathbf{u}, \boldsymbol{\theta}_{\text{kin}}), \tag{13}$$

$$\frac{\mathrm{d}T}{\mathrm{d}t} = \mathrm{Linear}\left(\frac{\mathrm{d}\widetilde{\mathbf{u}}}{\mathrm{d}t}, \boldsymbol{\theta}_{\text{thermo}}\right) + \epsilon, \tag{14}$$

and, after modeling the correction ϵ with a KAN,

$$\frac{\mathrm{d}T}{\mathrm{d}t} = \mathrm{Linear}\big(\mathrm{KAN}_{\text{kin}}(\mathbf{u}, \boldsymbol{\theta}_{\text{kin}}), \boldsymbol{\theta}_{\text{thermo}}\big) + \mathrm{KAN}_{\text{cor}}(\mathbf{u}, \boldsymbol{\theta}_{\text{cor}}). \tag{15}$$

The paper further defines the kinetic core and correction layer as

$$\mathrm{KAN}_{\text{kin}}(\mathbf{u}(t), \boldsymbol{\theta}_{\text{kin}}) = \left(\Psi_1^{\text{lean}} \circ \Psi_0^{\text{add}}\right)(\mathbf{u}(t)), \tag{16}$$

$$\mathrm{KAN}_{\text{cor}}(\mathbf{u}(t), \boldsymbol{\theta}_{\text{cor}}) = \Psi_0^{\text{add}}(\mathbf{u}(t)). \tag{17}$$

The two-stage training process changes which state is integrated, not Eqs. (13)–(17). In Stage 1, only $\widetilde{\mathbf{u}}$ is integrated and temperature is supplied externally: it is constant for biodiesel and read from a dense observed/reference interpolant for hydrogen and methane. In Stage 2, the complete state $\mathbf{u} = [\widetilde{\mathbf{u}}, T]$ is integrated and the kinetic and thermodynamic parameter groups are optimized together.

Loss. The printed ChemKAN objective is [8]

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= \mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{PINN}}(\boldsymbol{\theta}) \\ &= \frac{1}{n^*} \sum_{j=1}^{N_t} \sum_{k=1}^{n^*} \left(\widehat{u}_k^{\text{pred}}(t_j, \boldsymbol{\theta}) - \widehat{u}_k^{\text{obs}}(t_j)\right)^2 \\ &\quad + \alpha_{\text{PINN}} \sum_{i=1}^{N_e} \sum_{j=1}^{N_t} \left| \sum_{k=1}^{m} \frac{N_i^k W_i \left(Y_{k,j}^{\text{pred}} - Y_{k,1}^{\text{pred}}\right)}{W_k} \right|,\end{aligned} \tag{18}$$

where

$$n^* = \begin{cases} m, & \text{Stage 1,} \\ m+1, & \text{Stage 2.} \end{cases} \tag{19}$$

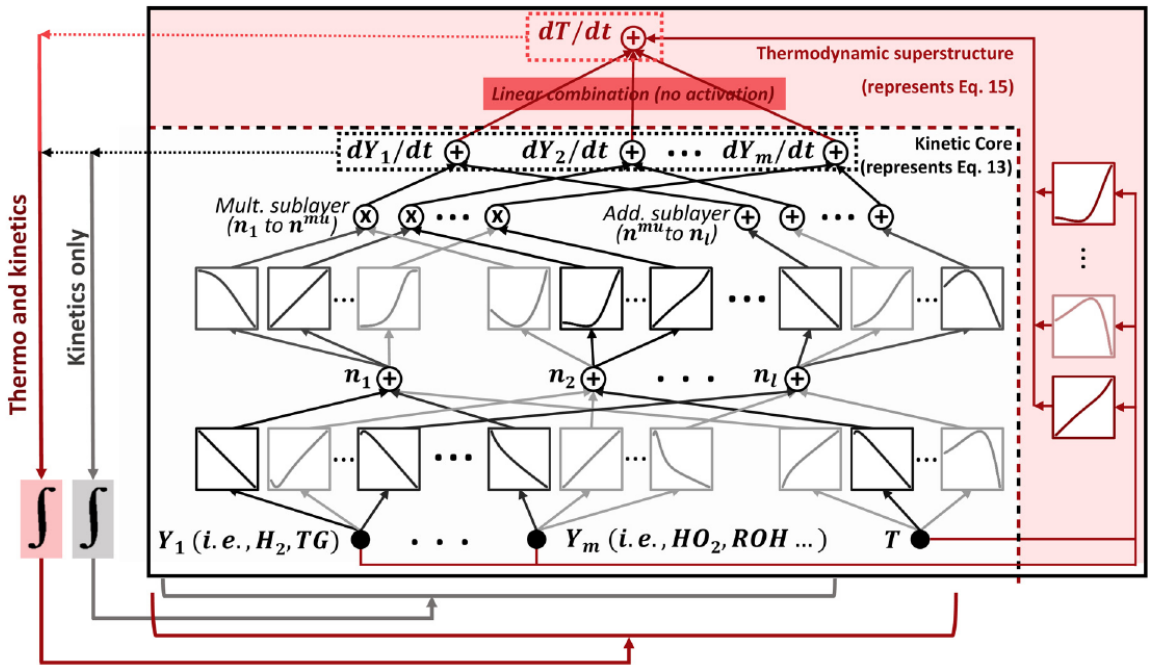


Figure 1: ChemKAN architecture proposed in Figure 2 of Koenig et al. [8]. The grey kinetic core maps the thermochemical state to species rates. The red thermodynamic superstructure combines these rates linearly and adds a correction KAN to obtain the temperature rate. The source article is published under CC BY-NC 4.0.

The conservation term is evaluated at every time point for every conserved element and weighted by $\alpha_{\text{PINN}} = 10^{-4}$. The paper uses only the MSE term for biodiesel, while hydrogen uses the MSE plus the element-conservation term [8]. The methane extension follows the hydrogen-style loss. The printed MSE averages over modeled states but sums over time. For reporting, this project also uses

$$\text{MSE}_{\text{time-avg}} = \frac{\mathcal{L}_{\text{MSE}}}{N_t}, \tag{20}$$

The released Julia biodiesel example instead uses `Flux.mse`, which averages over both state variables and observation times. The time-averaged convention is therefore used for numerical comparisons in this report, while the original training objective is retained where required for reproduction. Each figure and table identifies the loss convention used for its results.

2.3 Data generation and preparation

2.3.1 Biodiesel production

The biodiesel benchmark is an isothermal six-species kinetic system. The paper uses activation energies $E_a = [14.54, 6.47, 14.42]$ kcal mol^{-1} and $\ln A = [18.60, 7.93, 19.13]$ for reactions 1–3, respectively [8]. Temperatures are randomly sampled from 323–343 K, initial TG and ROH concentrations are sampled uniformly from 0.5–2, all other species start at zero, and 20 training plus 10 test trajectories are generated over 30 s with 30 sampled times [8].

A source-level discrepancy exists between the printed reaction ordering and the released biodiesel example. The released code uses $E_a[\cdot, \mathbf{14.42}, \mathbf{6.47}]$ kcal mol^{-1} and $\ln A = [\cdot, \mathbf{19.13}, \mathbf{7.93}]$. Controlled datasets and training runs were produced for both orderings. The released-code ordering more closely reproduces the species levels shown in the paper's Figure 3: DG reaches about 0.47 and MG about 0.17, consistent with the figure's 0–0.6 and 0–0.2 axes, whereas the paper-text ordering gives about 0.29 and 0.325, so the MG curve would exceed its axis. The main author-matched biodiesel reproduction therefore uses the released-code ordering and keeps the paper-order experiment separate [4].

The paper's Figure 3 and Figure 6 unseen condition is

$$[\text{TG}]_0 = 1.94, \quad [\text{ROH}]_0 = 1.43, \quad [\text{DG}]_0 = [\text{MG}]_0 = [\text{GL}]_0 = [\text{R}'\text{CO}_2\text{R}]_0 = 0, \quad T = 334.8\,\text{K}. \tag{21}$$

The main biodiesel figure runs use `biodiesel_v2.npz`, which was generated in Python from the released-code kinetics and the authors' initial conditions using SciPy LSODA with $\texttt{rtol} = 10^{-10}$ and $\texttt{atol} = 10^{-12}$. These trajectories are used as the training targets. PyTorch ChemKAN training uses `torchdiffeq`'s Tsit5 solver with $\texttt{rtol} = 10^{-2}$ and $\texttt{atol} = 10^{-6}$. For the biodiesel figure pipeline, the final-checkpoint losses used in Figures 4 and 5A and the trajectories used in Figures 3 and 6 are re-evaluated in double precision with Tsit5 at $\texttt{rtol} = 10^{-10}$ and $\texttt{atol} = 10^{-12}$, so that the reported model error is not dominated by the deliberately loose training solve. The per-epoch histories in Figure 5B retain the training-solver evaluations.

A separate implementation-comparison experiment uses training arrays exported from the released Julia example. For that dataset, an independent Python reconstruction using SciPy's LSODA solver [16] with $\texttt{rtol} = 10^{-12}$ and $\texttt{atol} = 10^{-14}$ serves only as a cross-check and is not used as a training target. The released Julia example uses `AutoTsit5(Rosenbrock23)` for both reference-data generation and ChemKAN integration. Reference trajectories are generated from the known Arrhenius reaction equations using the OrdinaryDiffEq default tolerances (reltol $= 10^{-3}$, abstol $= 10^{-6}$). During training, ChemKAN-predicted derivatives are integrated with reltol $= 10^{-2}$ and abstol $= 10^{-6}$, and the resulting trajectories are compared against the reference data. The same settings are used for validation and plotting, without a separate evaluation tolerance.

2.3.2 Hydrogen–air combustion

The hydrogen data were generated with Cantera [5] using `h2o2.yaml`. Argon was omitted, leaving nine tracked species mass fractions plus temperature. The Cantera reactor integrations use $\texttt{rtol} = 10^{-9}$ and $\texttt{atol} = 10^{-15}$. The design contains 36 combinations of $T_0 \in \{950, 1000, 1050, 1100, 1150, 1200\}$ K and $\phi \in \{0.5, 0.7, 0.9, 1.1, 1.3, 1.5\}$; 35 are used for training, while $(T_0 = 1150\,\mathrm{K}, \phi = 1.3)$ is held out. The printed paper omits $\phi = 0.5$ from its list despite stating that 36 combinations were used; the authors later clarified that 0.5 belongs to the grid [8, 2]. The local training dataset contains 100 uniformly spaced output times over the 0.6-ms trajectory, and a separately generated 20,000-point Cantera temperature cache supplies the linear temperature interpolant used during Stage 1. The authors clarified that their original trajectories were instead generated with Arrhenius.jl using CVODE_BDF and checked against Cantera; direct Cantera generation is therefore a choice of this reproduction rather than a claim about the authors' original pipeline [2].

The final author-matched architecture has hidden width 3, five RBF basis functions per edge, $n_{\mathrm{mu}} = 3$, no base activation, and 344 trainable parameters. Both training stages use Adam with learning rate 2×10^{-3} and Tsit5 with $\texttt{rtol} = 10^{-6}$ and $\texttt{atol} = 10^{-8}$. The PyTorch implementation integrates min–max-normalized states while the ODE solver advances physical time t in seconds. A reference time scale t_{ref} is used to scale the complete raw ChemKAN right-hand side:

$$\frac{d\hat{\mathbf{u}}}{dt} = \frac{F(\hat{\mathbf{u}}, \boldsymbol{\theta})}{t_{\mathrm{ref}}}$$

This scaling is analogous to the released biodiesel example, which divides its learned right-hand side by a constant factor of 50, although its trajectory spans only 30 s. For hydrogen, we instead choose the full 0.6-ms trajectory horizon as t_{ref}. Because no hydrogen source code was released, both this scaling and the value of t_{ref} are reported as PyTorch-reproduction choices rather than as author-specified settings.

2.3.3 Methane-air oxidation extension

Methane is not part of the original ChemKAN paper and is treated as a transfer experiment rather than a reproduction target. The extension uses the GRI-Mech 3.0 chemical mechanism [15]. Argon is omitted because it remains a constant-zero inert species in the selected setup, leaving 52 species plus temperature and a 1,892-parameter ChemKAN. The initial-condition grid mirrors the hydrogen design: $T_0 \in \{1400, 1450, 1500, 1550, 1600, 1650\}$ K and $\phi \in \{0.5, 0.7, 0.9, 1.1, 1.3, 1.5\}$, with $(T_0 = 1550\,\mathrm{K}, \phi = 1.3)$ held out. Each trajectory contains 100 observation times over 5 ms, and a 20,000-point temperature cache is used in Stage 1. Reference trajectories were generated with Cantera using an adiabatic constant-pressure reactor at an initial pressure of 1 atm.

The model uses the same author-matched architecture pattern and loss as hydrogen. The normalized trajectory coordinate uses $t_{\mathrm{ref}} = 5$ ms $= 5 \times 10^{-3}$ s, equal to the full methane trajectory horizon, with the normalized

right-hand side divided by t_{ref} while the solver advances physical time in seconds. This follows the same PyTorch normalized-time formulation as the hydrogen reproduction.

The methane experiment followed the same two-stage training formulation as hydrogen. Stage 1 continued to approximately 54,800 epochs, and its archived epoch-50,000 checkpoint was used to initialize Stage 2, which was trained for 200,000 epochs. Training used Adam with learning rate 2×10^{-3}, the Tsit5 solver with $\mathrm{rtol} = 10^{-6}$ and $\mathrm{atol} = 10^{-8}$, float32 precision, and direct automatic differentiation. The complete Stage-2 training history and checkpoints were preserved. Because optimization remained non-monotonic, archived checkpoints were independently evaluated over complete trajectories from the initial conditions. The primary evaluation model was selected using the mean time-averaged MSE across the 35 training conditions, without using the held-out condition for checkpoint selection. The selected checkpoint corresponds to Stage-2 epoch 180,000 because epoch 200,000 coincided with a spike. The final epoch-200,000 checkpoint was retained and evaluated separately.

2.4 Experiment design and reproducibility controls

The following controls were used throughout the project:

- Predefined reproduction targets were mapped to the original paper's figures and table (biodiesel Figures 3–6; hydrogen Figures 7–8 and Table I).
- Legacy experiments were preserved instead of overwritten. Author-clarified experiments use separate directories and explicit provenance, preventing post-hoc relabeling.
- Every major run stores its configuration, training history, checkpoint(s), run log, data-file hash, and, for key checkpoints, SHA-256 hashes.
- Random seeds are fixed for controlled comparisons. Single-seed findings are reported as single-run evidence rather than population-level claims.
- Train/test preprocessing is documented explicitly. Where released code uses a different convention from the paper, both conventions are recorded rather than silently reconciled.
- Evaluation code is separated from training: saved checkpoints are re-evaluated on fixed reference datasets and common diagnostic grids.
- Numerical sensitivity backends are validated independently before being used in long training runs.

The implementation, experiment configurations, selected checkpoints, evaluation artifacts, and computational environment are documented in the accompanying ChemKAN release repository. The repository also preserves the pinned 'torchdiffeq' source distribution, including its license and provenance information.

The main PyTorch reproduction uses the explicit Tsit5 integrator from 'torchdiffeq' [3]. It does not automatically switch to a stiff solver. This differs from the released Julia biodiesel example, which uses 'AutoTsit5(Rosenbrock23)' and can switch from Tsit5 to Rosenbrock23 when stiffness is detected [4]. The printed ChemKAN paper states Tsit5 for the ChemKAN/FSA solve [8].

2.5 Forward Sensitivity Analysis

The proposal specified forward sensitivity analysis (FSA) for propagating parameter derivatives through the ODE solver. A continuous FSA backend was therefore implemented in addition to direct automatic differentiation. For an integrated state $\mathbf{x}(t)$,

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = F(\mathbf{x}, t, \boldsymbol{\theta}), \tag{22}$$

and parameter vector $\boldsymbol{\theta}$, define

$$S(t) = \frac{\partial \mathbf{x}(t)}{\partial \boldsymbol{\theta}}. \tag{23}$$

Differentiating the state equation with respect to $\boldsymbol{\theta}$ gives the standard forward-sensitivity system [14]

$$\frac{\mathrm{d}S}{\mathrm{d}t} = J_x S + J_{\boldsymbol{\theta}}, \qquad J_x = \frac{\partial F}{\partial \mathbf{x}}, \qquad J_{\boldsymbol{\theta}} = \frac{\partial F}{\partial \boldsymbol{\theta}}, \qquad S(t_0) = 0. \tag{24}$$

The initial sensitivity is zero because the reactor initial condition does not depend on the learnable network parameters.

The solver advances the physical state and the sensitivity matrix simultaneously. In compact notation,

$$\mathbf{z}(t) = \big(\mathbf{x}(t), S(t)\big), \tag{25}$$

with

$$\frac{\mathrm{d}\mathbf{z}}{\mathrm{d}t} = \big(F(\mathbf{x}, t, \boldsymbol{\theta}),\ J_x S + J_{\boldsymbol{\theta}}\big)\,. \tag{26}$$

Here $\mathrm{d}\mathbf{z}/\mathrm{d}t$ is only the time derivative of the augmented state. It is not a derivative of $\mathbf{z}$ with respect to $\mathbf{x}$. In code, the physical state and sensitivity matrix are combined into a single augmented tensor for ODE integration; mathematically, S remains the state-by-parameter sensitivity matrix.

In PyTorch, the existing `KineticDynamics`/`ChemKANDynamics` right-hand side is called through `torch.func.functional_call`; the model dynamics are therefore not reimplemented for FSA. `torch.func.jacrev` forms the state and parameter Jacobians, and `torch.func.vmap` applies the calculation independently over trajectories in a batch. The augmented system is integrated with the same run-level solver and tolerances. After integration, the parameter gradient is assembled as

$$\frac{\mathrm{d}L}{\mathrm{d}\boldsymbol{\theta}} = \sum_j S(t_j)^{\mathsf{T}} \frac{\partial L}{\partial \mathbf{x}(t_j)}. \tag{27}$$

For Stage 1, $\mathbf{x} = \widetilde{\mathbf{u}}$ and temperature is an external input. For Stage 2, $\mathbf{x} = \mathbf{u}$, so J_x automatically carries species–temperature coupling into the sensitivity equations.

The FSA implementation integrates the physical state and its parameter sensitivities as a single augmented system. The adaptive Tsit5 solver applies the specified relative and absolute tolerances to the entire augmented state using its default error norm. In contrast, direct differentiation integrates only the physical state and differentiates the numerical solver computation.

Consequently, the two approaches may use different adaptive integration steps even when their nominal solver tolerances are identical. This distinction is considered when comparing training behavior, but it does not imply that either differentiation backend is universally preferable.

2.6 Evaluation metrics and baselines

The proposal defined training/test MSE, generalization to held-out initial conditions, noise-free test error under noisy training, and inference time as the primary metrics. The report retains those metrics and adds combustion-specific diagnostics where a single aggregate MSE can hide physically important behavior.

Table 2: Primary evaluation metrics.

Metric	**Purpose**
Trajectory MSE	Primary normalized prediction error. The printed Eq. (18) averages over state variables and sums over time. For cross-plot reporting, $\mathrm{MSE}_{\text{time-avg}} = \mathcal{L}_{\mathrm{MSE}}/N_t$ is also reported. The released biodiesel Julia example uses this time-averaged form through `Flux.mse`; hydrogen training retains Eq. (18).
Generalization	Held-out initial-condition error; for hydrogen, a reconstructed 21×21 grid (441 conditions) is also evaluated.
Noise robustness	Clean-ground-truth test MSE after training on noisy observations; train-loss increase is compared with the irreducible noise floor.
Ignition behavior	Temperature rise, ignition delay defined as $\arg\max_t \mathrm{d}T/\mathrm{d}t$, and integration failures.
Complexity	Trainable parameter count and model organization.
Efficiency	End-to-end trajectory integration runtime under a common task definition; speed-up is traditional-solver runtime divided by ChemKAN runtime.

For combustion, ignition delay is defined as the time of maximum temperature-rise rate, following the original ChemKAN study:

$$t_{\mathrm{ign}} = \arg\max_t \frac{\mathrm{d}T}{\mathrm{d}t}. \tag{28}$$

Numerically, the temperature derivative is estimated on the diagnostic time grid using finite differences. For initial condition i, the absolute relative ignition-delay error is

$$e_{\text{ign}}^{(i)} = \frac{\left| t_{\text{ign,pred}}^{(i)} - t_{\text{ign,ref}}^{(i)} \right|}{t_{\text{ign,ref}}^{(i)}}. \tag{29}$$

The reported median absolute relative error is 100% times the median of $e_{\text{ign}}^{(i)}$ across the evaluated conditions. For the hydrogen Figure 8B comparison, this comprises 30 initial conditions.

DeepONet is reimplemented for the biodiesel architectural comparison. For combustion, ChemNODE is used as a literature baseline, with its reported results taken from the original ChemNODE study [12] rather than from an independent reimplementation. These published results provide context for the comparison but are not treated as direct same-hardware measurements. Traditional numerical references are Cantera for combustion and direct integration of the known biodiesel kinetics.

Cantera has two distinct roles in the combustion experiments. First, it generates the detailed-chemistry reference trajectories against which ChemKAN accuracy is measured. Thus, Cantera is not a competing predictor in the accuracy metric; ChemKAN error is measured relative to the Cantera solution. Second, computational efficiency is evaluated separately by re-running the *same initial-value problem* with Cantera and the trained ChemKAN model on identical hardware, with model/mechanism loading outside the timed region. The speed-up is defined as

$$S_{\text{speed}} = \frac{t_{\text{traditional}}}{t_{\text{ChemKAN}}}. \tag{30}$$

A value greater than one indicates acceleration. Accuracy and runtime are reported together, because a faster surrogate is only useful if the loss of fidelity is acceptable.

3 Results and Evaluation

3.1 Biodiesel reproduction

The initial biodiesel reproduction was based on the paper alone and required a substantial series of ablations. Several qualitative dynamics could be learned, but the published loss levels and robustness trends were not reproduced consistently. One early assumption was multiplicative observation noise. The author clarification later specified additive Gaussian noise of the form $\mathcal{N}(0, 1)$ times each species maximum and the requested noise percentage, clipped at zero; the clarified dataset and final reproduction use this additive construction [2]. The released Julia example further established the no-base 156-parameter ChemKAN and exposed implementation details that differ from the printed paper [4].

A controlled reaction-order experiment used the same initial conditions, training settings, and seed while changing only the reaction-2/reaction-3 Arrhenius ordering. Aggregate losses were of similar scale, but the released-code ground truth produced DG and MG levels that match the y-axis ranges and trajectories of the published Figure 3 more closely. Because the authors did not explicitly confirm which ordering generated the published figure, this is reported as evidence rather than as a correction of the paper.

The strongest quantitative reproduction is the noise-floor behavior. Under the authors' released-example time-averaged loss, the $0\rightarrow1\%$ and $0\rightarrow5\%$ training-loss increments are close to the values stated in the paper, and the 15% clean-test degradation is close to the reported approximately two-fold increase. Neural-scaling exponents are less stable across the reconstructed model-size set and the single training seed.

The DeepONet architecture was reconstructed using the parameter count reported in the ChemKAN paper and the branch-input dimensions clarified by the authors. However, the exact activation functions, learning rate, and time-input preprocessing were not fully specified. An initial ReLU-based implementation did not reproduce the reported predictive behavior satisfactorily. Consequently, additional controlled ablations were conducted to investigate activation functions and optimization settings. The selected reconstruction uses tanh activations, raw physical time as the trunk input, and an Adam learning rate of 10^{-2}. This configuration is reported as the selected behavior-matching reconstruction rather than an exactly recovered author implementation. The original ReLU experiments are retained as diagnostic evidence.

Table 3: Biodiesel reproduction results compared with the original ChemKAN study. MSE values follow the time-averaged convention.

Quantity	**Original study**	**PyTorch reproduction**
ChemKAN neural-scaling exponent (train/test)	1.0 / 0.6	1.49 / 0.372
78-parameter ChemKAN error (train/test)	$\sim 10^{-4}$	4.79×10^{-4} / 7.68×10^{-4}
Training-loss increase, 0–1% noise	3.78×10^{-5}	4.19×10^{-5}
Training-loss increase, 0–5% noise	9.64×10^{-4}	1.05×10^{-3}
ChemKAN Clean-test error ratio, 15% / 0%	~ 2.0	1.84
15% clean-test error ratio, DeepONet / ChemKAN	4.4	2.73

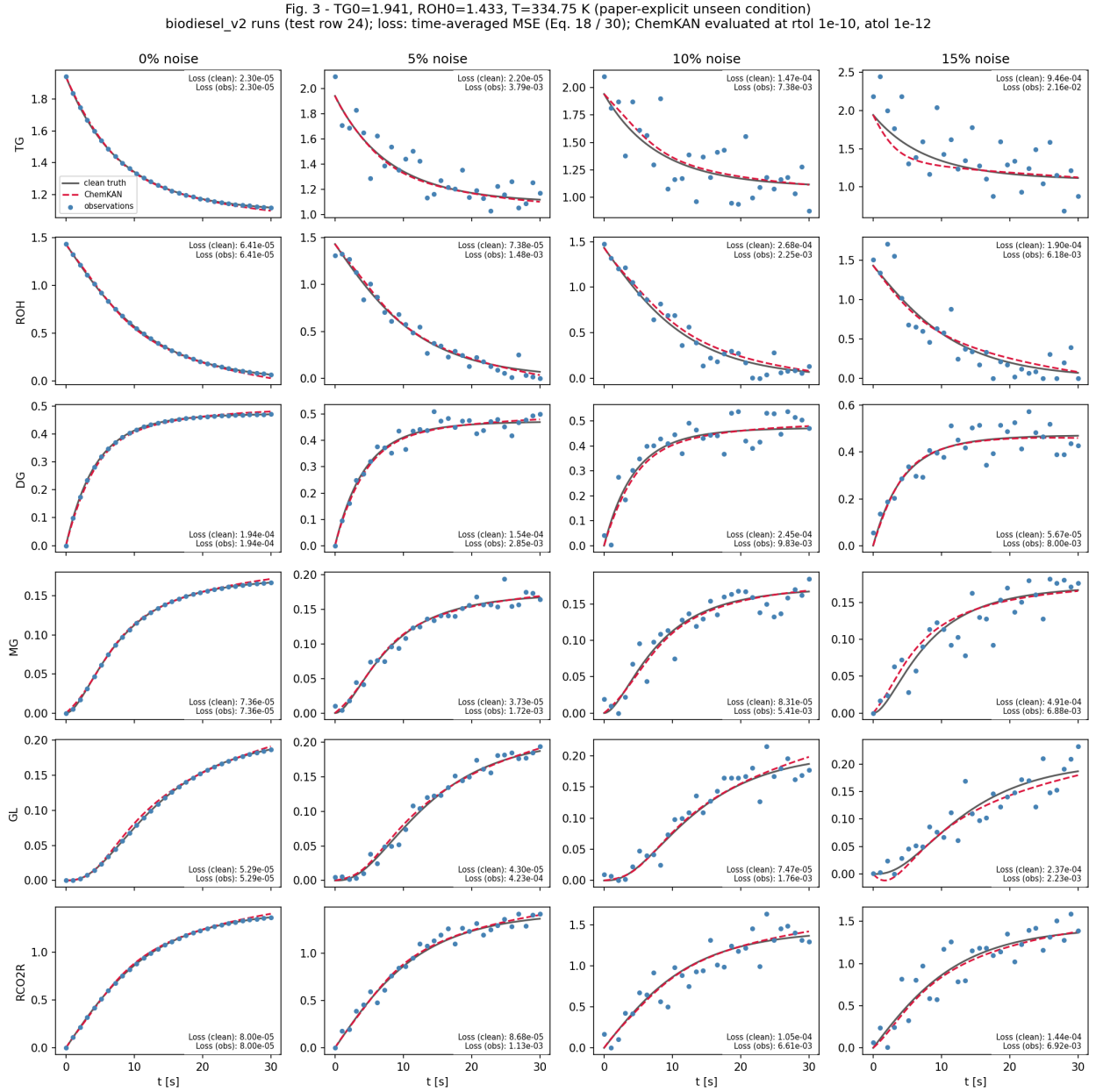


Figure 2: ChemKAN reproduction of the paper's Figure 3 unseen condition under 0%, 5%, 10%, and 15% observation noise. The initial condition is $[\mathrm{TG}]_0 = 1.94$, $[\mathrm{ROH}]_0 = 1.43$, all four other species initially zero, and $T = 334.8$ K. The model was trained with Tsit5 at $\texttt{rtol} = 10^{-2}$ and $\texttt{atol} = 10^{-6}$; the displayed curves were re-evaluated at $\texttt{rtol} = 10^{-10}$ and $\texttt{atol} = 10^{-12}$. Each species uses its own y-axis scale, following the paper's presentation.

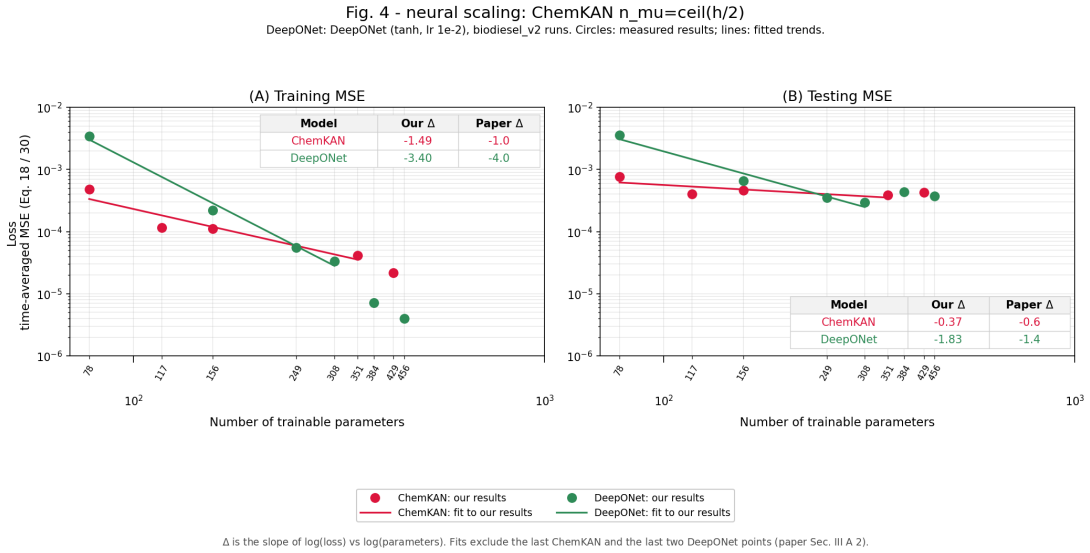


Figure 3: Neural scaling on noise-free biodiesel data (seed 0). Final-checkpoint time-averaged normalized training and testing MSE are shown for ChemKAN models trained for 5,000 epochs and the selected tanh DeepONet reconstruction trained for 50,000 epochs. Log–log fits exclude the 429-parameter ChemKAN and the 384- and 456-parameter DeepONets, following the saturation exclusions described in the original study. ChemKAN predictions are evaluated with $\mathrm{rtol} = 10^{-10}$ and $\mathrm{atol} = 10^{-12}$.

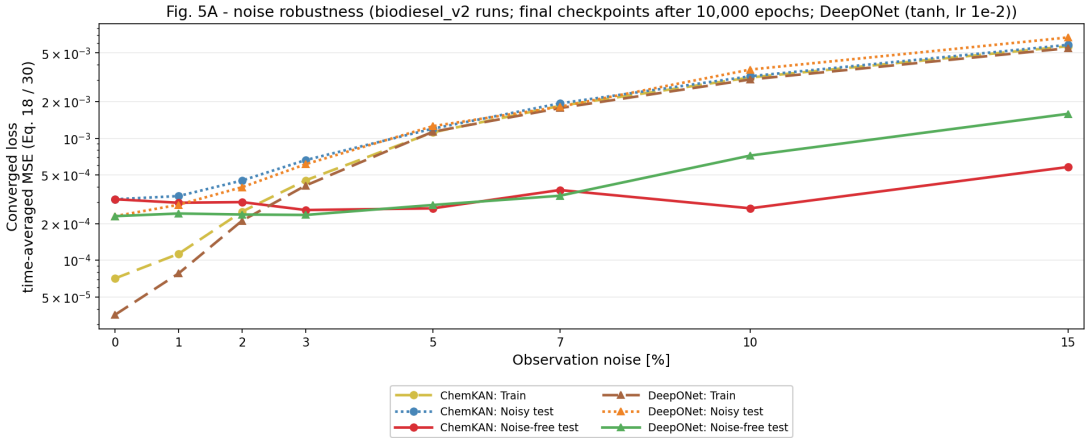


Figure 4: Noise robustness of the 156-parameter ChemKAN and 308-parameter tanh DeepONet after 10,000 training epochs (seed 0). The curves show time-averaged normalized MSE against noisy training observations, noisy held-out observations, and clean held-out reference trajectories. The evaluation uses the noise-level-specific normalization recorded in the experiments. ChemKAN predictions are evaluated with $\mathrm{rtol} = 10^{-10}$ and $\mathrm{atol} = 10^{-12}$.

Figure 4 compares the final checkpoints of the 156-parameter ChemKAN and the selected 308-parameter tanh DeepONet reconstruction, each trained for 10,000 epochs with seed 0 at the corresponding observation-noise level. The DeepONet uses raw physical time as its trunk input and an Adam learning rate of 10^{-2}.

At lower noise levels, the selected DeepONet achieved comparable or lower clean-test MSE in several cases. At 10% and 15% noise, ChemKAN achieved approximately 2.7-fold lower clean-test MSE. The 15% ratio was 2.73, compared with 4.4 reported in the original study. These single-seed results support a ChemKAN advantage at higher noise levels, but not uniform superiority across all tested noise levels.

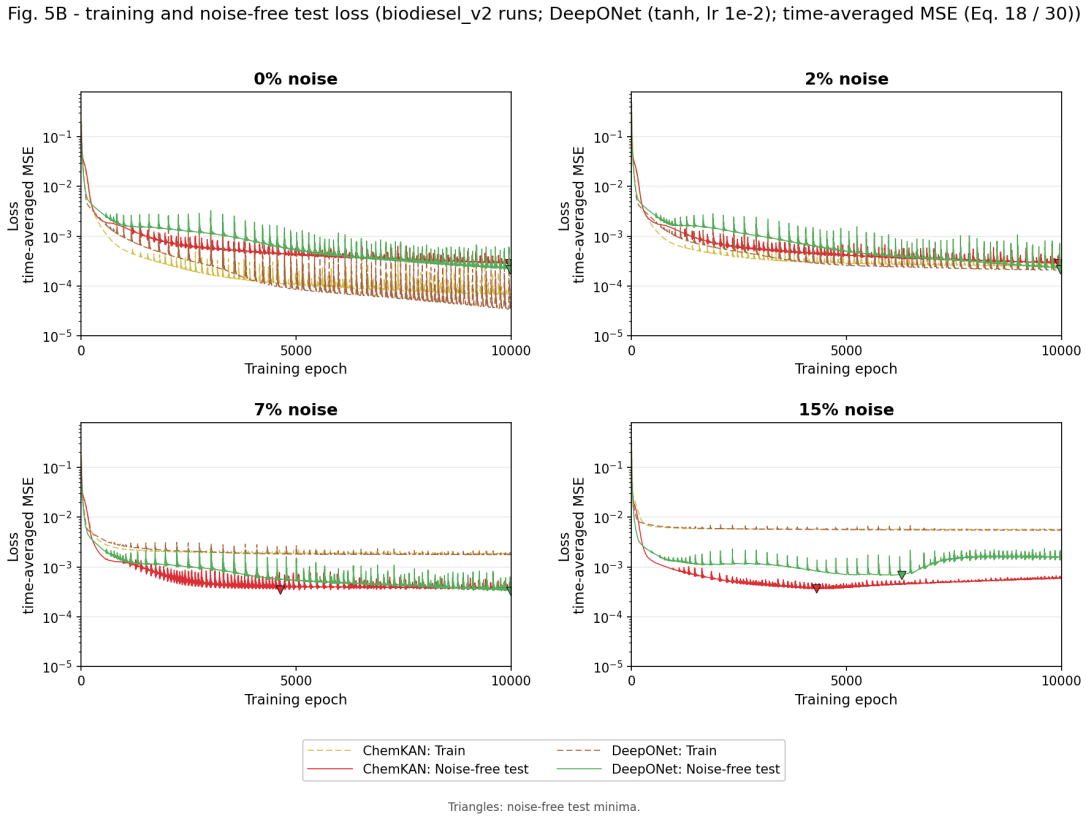


Figure 5: Recorded training and clean-test MSE histories of ChemKAN and the selected tanh DeepONet at 0%, 2%, 7%, and 15% observation noise. MSE is averaged over species and observation times. The ChemKAN histories use the training-solver tolerances $\mathrm{rtol} = 10^{-2}$ and $\mathrm{atol} = 10^{-6}$. Triangles indicate the minimum recorded clean-test MSE.

3.2 Hydrogen-air combustion reproduction

Hydrogen was the most difficult reproduction target. Under the initial paper-only protocol, the species trajectories could be learned to a useful qualitative level, but the Stage-2 temperature dynamics often failed to ignite and the loss remained high. Diagnostics showed that the thermodynamic linear layer initially produced a temperature derivative with insufficient magnitude. The author clarification supplied several missing settings, including no base activation, five RBF basis functions, min–max-normalized inputs, thermodynamic linear weights initialized to 10^{-5}, a 100-point training grid, a separate dense temperature interpolant, and a 10^5–2×10^5 epoch range per stage [2].

The normalized-state formulation also changes the scale of the ODE right-hand side: dividing the complete normalized RHS by the 0.6 ms trajectory horizon increases its physical-time magnitude by $1/t_{\mathrm{ref}}$. Together with the clarified architecture and initialization, this removed the earlier temperature-rate magnitude mismatch and allowed the model to learn ignition. The individual causal contribution of the time scaling versus the other clarified changes was not isolated in a single-factor ablation, so the report does not attribute the recovery to one change alone.

The main hydrogen reproduction used 100,000 Stage-1 epochs followed by 200,000 Stage-2 epochs. Stage 1 exhibited non-monotonic optimization behavior, with its lowest logged time-averaged MSE of 5.334×10^{-4} at

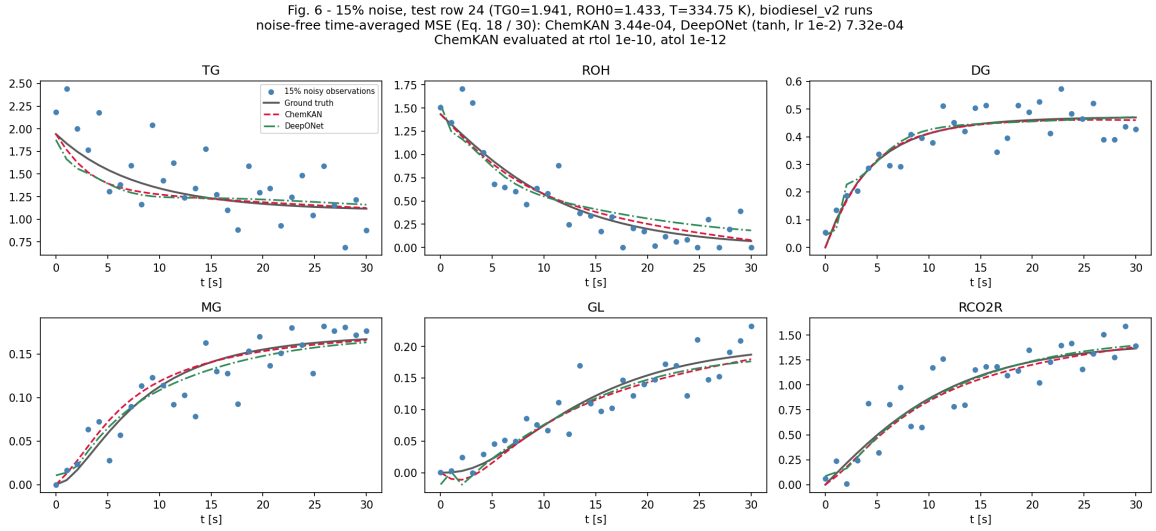


Figure 6: Biodiesel species trajectories for the unseen initial condition $[\mathrm{TG}]_0 = 1.940826$, $[\mathrm{ROH}]_0 = 1.433298$, and $T = 334.7486$ K, using ChemKAN and DeepONet models trained with 15% observation noise. The curves correspond to the final 156-parameter ChemKAN and 308-parameter tanh DeepONet checkpoints. Time-averaged clean-test MSE for this trajectory is 3.4408×10^{-4} for ChemKAN and 7.3175×10^{-4} for DeepONet.

epoch 60,037. The fixed 100k checkpoint was retained as the starting point for Stage 2. In Stage 2, the lowest logged time-averaged MSE was 4.548×10^{-4} at epoch 199,954, while the epoch-200,000 value was 6.360×10^{-4}. Logged training errors are distinguished from the independent evaluations of the saved checkpoints.

Table 4: Hydrogen Stage-2 training progression and independent checkpoint evaluation. Logged training MSE is recorded at the specified epoch, while held-out MSE is obtained by evaluating the corresponding checkpoint. Both use time-averaged normalized MSE over 100 time points. Ignition-delay error denotes the median absolute relative error over the 30 Figure 8B initial conditions.

Epoch	**Train MSE**	**Held-out MSE**	**Ignition-delay error**
100k	1.388×10^{-3}	8.170×10^{-4}	3.69%
150k	7.425×10^{-4}	4.526×10^{-4}	1.64%
200k	6.360×10^{-4}	3.507×10^{-4}	1.36%

The epoch-200,000 checkpoint was used for the main hydrogen evaluation as the endpoint of the planned Stage-2 training budget, consistent with the authors' reported training range. Its held-out MSE and ignition-delay error are reported alongside those of the earlier archived checkpoints to illustrate training progression.

A reconstructed 21×21 grid (441 conditions) was evaluated in float64 with Tsit5 at `rtol`$= 10^{-10}$ and `atol`$= 10^{-12}$. The time-averaged normalized trajectory MSE has median 4.91×10^{-4}, minimum 3.83×10^{-5}, and maximum 6.12×10^{-2}. Across the 30 Figure 8B, the median predicted temperature rise is 1604.90 K and the median Cantera reference rise is 1580.73 K. The difference between these two medians is 24.17 K (1.53% of the reference median); this is not the median of the paired temperature-rise errors. The median absolute ignition-delay relative error is 1.36%.

Figure 7 compares the reproduced trajectories at a training condition and the designated held-out condition. Figure 8 shows prediction errors across the reconstructed initial-condition grid, while Figure 9 compares ignition-delay behavior.

The dense-grid evaluation reveals substantial differences in accuracy across initial conditions. Although the median time-averaged MSE over all 441 conditions is 4.91×10^{-4}, errors increase considerably for some conditions near the ignition-transition region. For example, the median time-averaged MSE across equivalence ratios at $T_0 = 987.5$ K is 3.47×10^{-2}, compared with 4.89×10^{-4} at $T_0 = 1050$ K.

At $T_0 = 950$ K, where reference trajectories remain approximately isothermal over the observation window, the learned model produces small spurious temperature increases for some conditions, but the corresponding normalized trajectory errors remain comparatively low. These results indicate that predictive accuracy depends

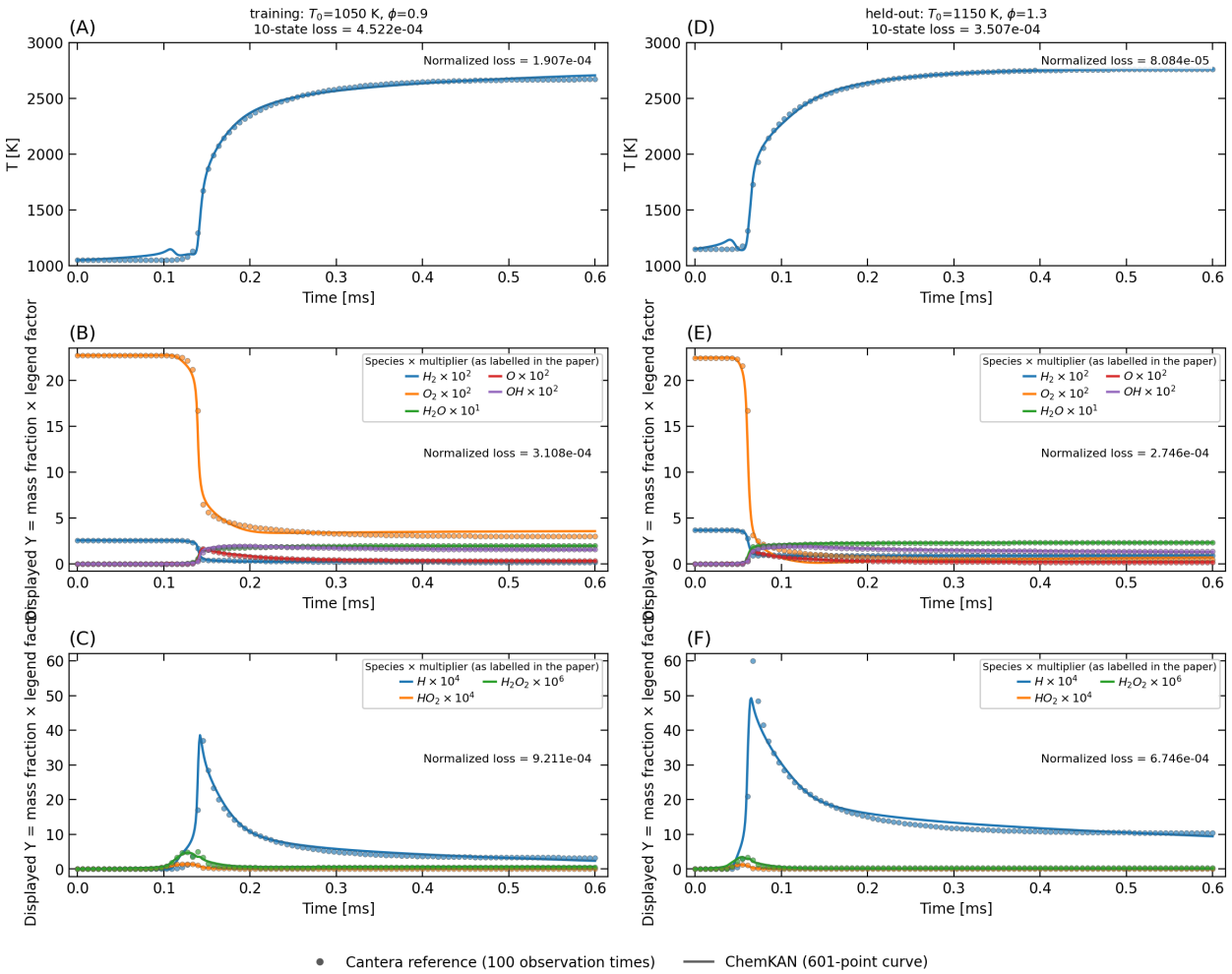


Figure 7: Hydrogen trajectories from the 200k Stage-2 checkpoint, re-evaluated in float64 with Tsit5 at `rtol`$= 10^{-10}$ and `atol`$= 10^{-12}$. The left column is the training condition $\phi = 0.9$, $T_0 = 1050$ K; the right column is the held-out condition $\phi = 1.3$, $T_0 = 1150$ K, corresponding to the paper's Figure 7.

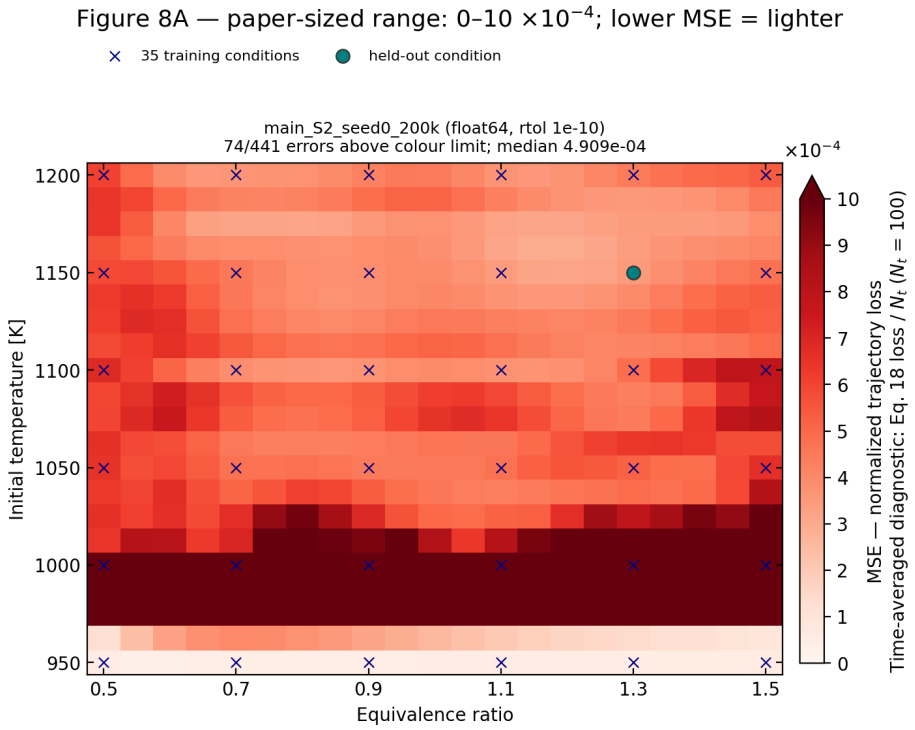


Figure 8: Dense 441-condition generalization map for the 200k Stage-2 checkpoint, evaluated in float64 with Tsit5 at `rtol`$= 10^{-10}$ and `atol`$= 10^{-12}$. The plotted value is the normalized trajectory Eq. (18) MSE divided by the 100 observation times.

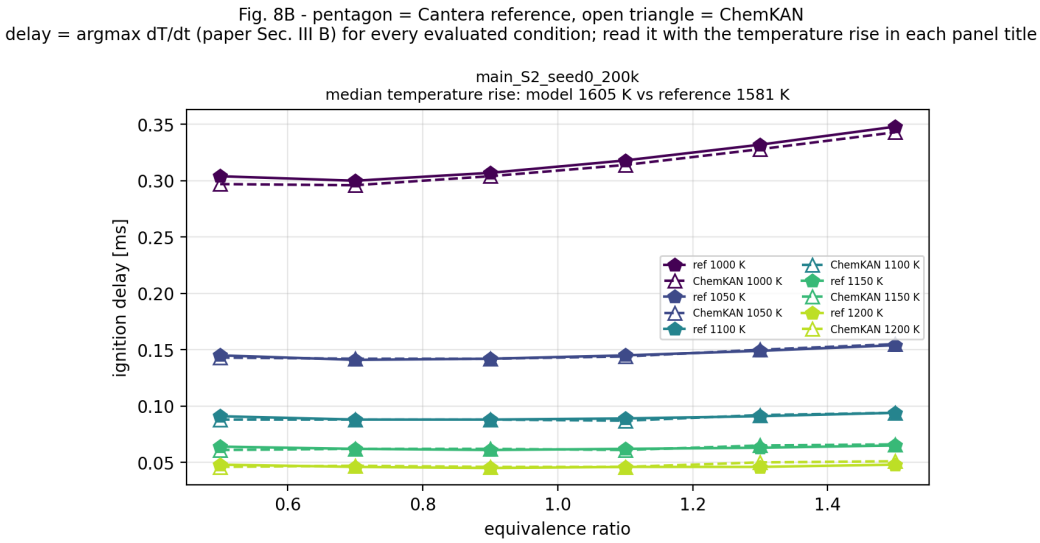


Figure 9: Ignition-delay comparison for the 200k Stage-2 checkpoint on the 30 conditions of the paper's Figure 8B ($T_0 = 1000, 1050, 1100, 1150, 1200$ K and all six equivalence ratios). Ignition delay is the time of maximum $\mathrm{d}T/\mathrm{d}t$.

on the initial thermochemical conditions. Further inspection of individual trajectories is required to distinguish ignition-timing errors from inaccuracies in temperature and species dynamics.

3.3 FSA validation and sensitivity-backend findings

FSA was validated before it was used for scientific training. The validation suite covers: (i) the analytic scalar problem $\mathrm{d}y/\mathrm{d}t = \theta y$, whose sensitivity is known in closed form; (ii) biodiesel at initialized and trained parameters; (iii) hydrogen Stage 1 at initialized and trained parameters; and (iv) hydrogen Stage 2 at initialized and trained checkpoints under the tested thermodynamic initializations. It checks functional equivalence, FSA-versus-direct-autograd gradients under tolerance refinement, physical-state consistency, trajectory independence, production dtype, restart behavior, and selected finite-difference probes. The forward-sensitivity equation follows the standard formulation documented by SciMLSensitivity [14].

Table 5: Forward Sensitivity Analysis validation summary.

Validation item	**Status**	**Meaning**
Analytic ODE test	PASS	At tight tolerances, state/sensitivity and loss-gradient relative errors reach approximately 10^{-9}.
Functional equivalence	PASS	The functional dynamics used for Jacobians reproduce the existing dynamics wrapper to numerical precision.
Gradient comparison	PASS under extended tolerance refinement	One hydrogen Stage-1 initial case required extending the tolerance ladder from $10^{-10}/10^{-12}$ to $10^{-12}/10^{-14}$; at the finer level both FSA and direct autograd converged and agreed under the same pre-defined thresholds.
State consistency / independence	PASS after refinement amendment	The augmented solve reproduces the physical state, while batch-independence checks passed at the amended finer tolerance level.
Production dtype / resume	PASS	Float32 production runs remain finite and interrupted FSA runs reproduce uninterrupted optimizer/model/RNG state.
Finite differences	INCONCLUSIVE (optional)	Four selected probes pass; one very small correction-KAN gradient is dominated by cancellation and is not used as a blocking test.

The validation results support the correctness of the implemented continuous FSA backend under the tested configurations. The gradient-comparison and batch-independence checks required an additional solver-tolerance refinement level (down to `rtol` $= 10^{-12}$ and `atol` $= 10^{-14}$ in the affected comparison), as recorded in the historical validation log. Some cases did not pass on the original refinement ladder. The pre-defined gradient-error acceptance thresholds were unchanged.

A controlled biodiesel experiment provides an additional implementation-level comparison with the authors' released Julia example. Starting from the exact Julia initialization and using the same released-example data/settings, PyTorch FSA gives Julia-relative training-loss ratios of approximately 1.0001, 0.9999, and 0.997 at epochs 500, 2,000, and 5,000; the late training median is 0.94× Julia and the late validation median is 1.19× Julia. Direct differentiation through the PyTorch state-only solve remains about 1.5× above the Julia training loss in the same controlled comparison.

The released Julia biodiesel example uses `ForwardDiff.gradient` through `AutoTsit5(Rosenbrock23)`, so derivative information participates in the differentiated solver computation. PyTorch FSA likewise integrates the sensitivity components with the physical state, whereas direct autograd integrates only the physical state and differentiates the solver computation graph afterwards. This offers a plausible numerical explanation for the closer PyTorch-FSA/Julia trajectory, but the project treats it as a hypothesis rather than a demonstrated causal mechanism.

The main hydrogen reproduction used direct automatic differentiation, while the FSA backend was validated independently and evaluated in the controlled biodiesel comparison.

3.4 Methane extension

The methane-air oxidation experiment investigated the transfer of ChemKAN to a larger chemical system not included in the original study [8]. The model represented 52 species and temperature using GRI-Mech 3.0, with 1,892 trainable parameters.

The methane Stage-1 training was initially conducted on TU Wien's JupyterHub and resumed on a local machine after epoch 28,000. Stage 1 continued to approximately 54,800 epochs, and the archived epoch-50,000 checkpoint was used to initialize Stage 2. Stage 2 was subsequently trained for 200,000 epochs. Because optimization was non-monotonic, archived checkpoints were evaluated through complete trajectory integrations. The epoch-180,000 checkpoint was selected using the mean time-averaged MSE on the 35 training conditions, without using the held-out condition. The final epoch-200,000 checkpoint was retained for comparison.

The selected model achieved a time-averaged normalized MSE of 1.163×10^{-3} on the held-out condition ($T_0 = 1550$ K, $\phi = 1.3$). Across the complete 36-condition evaluation grid, the median time-averaged MSE was 9.94×10^{-4}. The model reproduced the overall ignition events and major species trends. However, some predicted temperature trajectories developed a two-stage rise where the reference temperature increased more abruptly. Errors were also more pronounced for certain radical and intermediate species.

The final epoch-200,000 checkpoint was evaluated separately and yielded a higher median time-averaged MSE of 1.47×10^{-3} across the 36 conditions, and 1.27×10^{-3} for the held-out condition. These results illustrate that longer training did not monotonically improve trajectory accuracy. The selected epoch-180,000 checkpoint is evaluated visually in Figure 10, which compares its held-out trajectory with the Cantera reference, and in Figure 11, which shows prediction errors across the broader 441-condition grid.

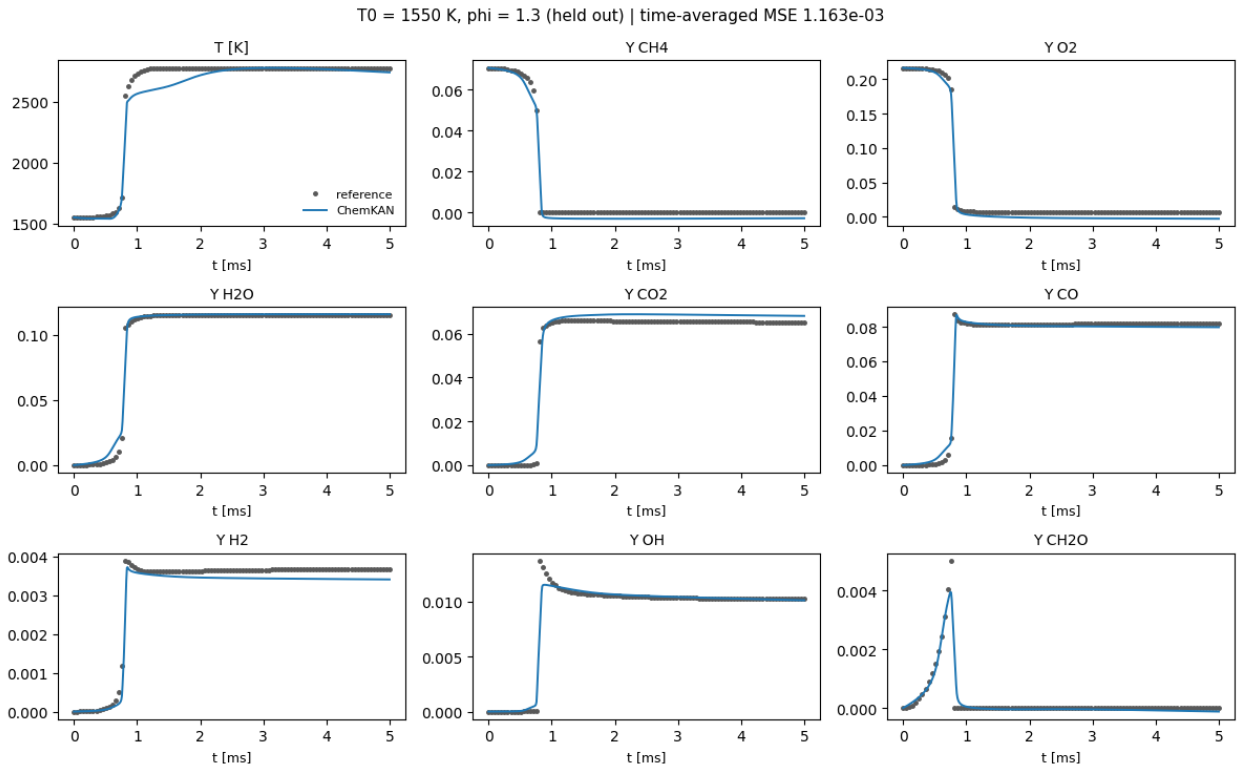
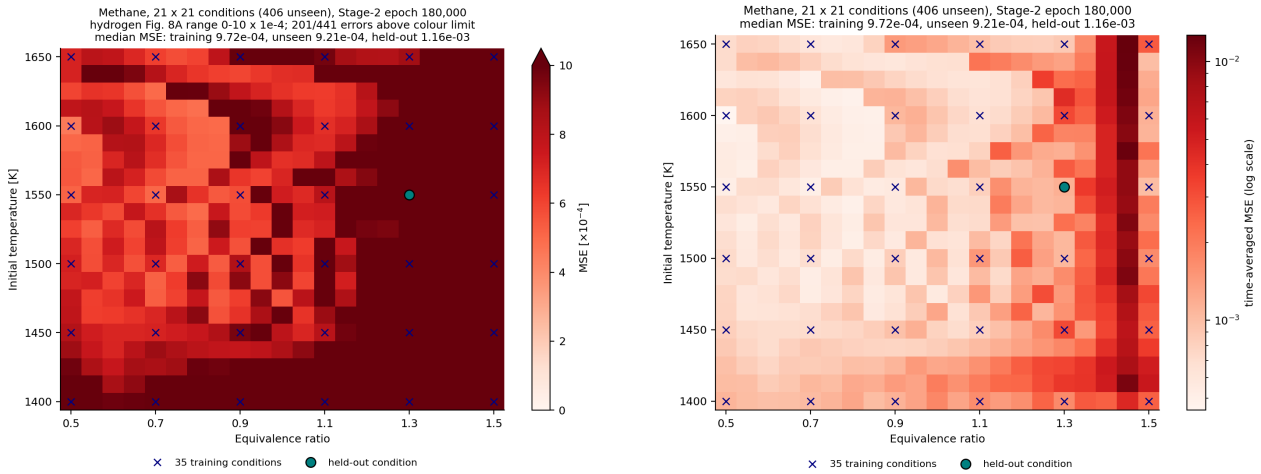


Figure 10: Methane-air oxidation on the held-out condition $T_0 = 1550$ K, $\phi = 1.3$, using the selected epoch-180,000 Stage-2 checkpoint. ChemKAN complete-rollout predictions are compared with Cantera reference observations for temperature and selected species. The time-averaged normalized MSE is 1.163×10^{-3}.

The broader 21×21 initial-condition grid contains 35 training and 406 unseen conditions. The median time-averaged MSE is 9.72×10^{-4} for the training conditions and 9.21×10^{-4} for the unseen conditions. Although the model generalizes across much of the investigated domain, higher errors occur around $\phi = 1.40$–1.45, where the maximum time-averaged MSE reaches 1.27×10^{-2}. The corresponding errors are particularly pronounced for several low-concentration species. These observations show that the model's generalization accuracy varies across the initial-condition domain, despite the similar median errors of the training and unseen groups.



(a) Paper-style linear color scale.

(b) Full-range logarithmic color scale.

Figure 11: Generalization of the selected methane Stage-2 epoch-180,000 checkpoint across 441 initial conditions, comprising 35 training conditions and 406 unseen conditions. Time-averaged normalized MSE was evaluated over 100 observation times using float32 Tsit5 with $\mathrm{rtol} = 10^{-6}$ and $\mathrm{atol} = 10^{-8}$. The left panel uses the paper-style color range up to 10^{-3}, while the right panel shows the full error range logarithmically.

Although Stage 2 was trained for 200,000 epochs, the optimization had not clearly converged. The median logged time-averaged MSE decreased from 1.785×10^{-3} over epochs 170,001–180,000 to 1.598×10^{-3} over epochs 190,001–200,000. Further optimization or alternative Stage-1 checkpoint selection might affect the final accuracy. These possibilities were not investigated systematically, and the 200k training budget was retained for consistency with the hydrogen experiments. The complete methane training history is provided in Appendix C.

3.5 Computational efficiency

A fair efficiency comparison must time the complete trajectory prediction task, not only a neural-network forward pass. ChemKAN still requires numerical ODE integration; it replaces the detailed chemical source-term function with a learned source-term model. For hydrogen and methane, the final benchmark therefore uses identical initial conditions, physical time horizons, and requested output grids for both Cantera and ChemKAN. Mechanism/model loading and data-file I/O are performed outside the timed region. Repeated measurements are used to report median latency, and accuracy is attached to every runtime result.

This is conceptually the same type of comparison used in ChemNODE. ChemNODE generated its reference/training trajectories with Cantera and then compared the time required to advance the learned model and the full detailed mechanism to steady state, reporting approximately $2.3\times$ speed-up [12]. The original ChemKAN study reported approximately $2\times$ acceleration against detailed chemistry implemented with Arrhenius.jl [8, 6]. The project-specific timing numbers are therefore interpreted as implementation- and hardware-specific measurements rather than as direct reproductions of those published speed-up factors.

Legacy local benchmarks were informative but are not used as the final efficiency claim because they were produced before the author-matched hydrogen configuration.

Benchmark implementation. Computational efficiency was measured for complete trajectory integration, including per-trajectory initialization and output conversion, while excluding model loading, mechanism loading, compilation and file I/O. The benchmark used one CPU thread on an Apple M1, three warm-up calls, and 25 timed repetitions. Single-trajectory latency and multi-condition throughput were measured separately.

In addition to the primary PyTorch implementation, an inference-only Julia port was developed with AI-assisted code generation using OrdinaryDiffEq.jl and the Tsit5 solver [13]. The Julia implementation was not trained or refitted; it used the same learned parameters and normalization as the PyTorch model. The two implementations were compared numerically to verify agreement of their predicted trajectories before the runtime benchmarks were conducted.

For hydrogen, the same 36 initial-value problems were solved over 0.6 ms at 100 output times. The detailed-chemistry references were Cantera and Arrhenius.jl using the corresponding hydrogen mechanism. For biodiesel, the learned dynamics were compared against the known reaction equations using SciPy LSODA and Julia-based numerical integration.

Table 6: Single-trajectory median inference latency on an Apple M1 CPU (one thread, 100 output points). The selected PyTorch-trained ChemKAN parameters were used in both execution environments. Detailed-chemistry references were evaluated at their data-generation tolerances.

System	PyTorch	Julia	Python ref.	Julia ref.
		Median latency (ms)		
Hydrogen	111.51	1.60	3.94	10.07
Methane	117.05	9.93	39.57	134.29
Biodiesel	67.17	0.69	1.20	0.20

The combustion ChemKAN integrations used $\mathrm{rtol} = 10^{-6}$ and $\mathrm{atol} = 10^{-8}$, whereas the detailed chemistry references used the tighter data-generation tolerances ($10^{-9}/10^{-15}$). The biodiesel comparison used its separately documented tight evaluation settings. Consequently, the measured speed-ups apply to these numerical configurations and do not constitute comparisons at guaranteed equal numerical error.

The implementation language substantially affected computational efficiency. For hydrogen, the Julia ChemKAN inference implementation achieved approximately $2.5\times$ lower latency than Cantera and $6.3\times$ lower latency than Arrhenius.jl. For methane, the corresponding speed-ups increased to approximately $4.0\times$ and $13.5\times$.

In contrast, PyTorch ChemKAN required approximately 111.5 ms and 117.0 ms per trajectory for hydrogen and methane, respectively, and was slower than Cantera in both cases.

The Julia implementation used weights exported from the PyTorch-trained models without retraining. Agreement between the two inference implementations was independently verified. Thus, the runtime differences are associated with the numerical and software execution environment rather than with different learned model parameters. For biodiesel, which contains only three reactions, Julia's direct integration of the known kinetics was faster than Julia ChemKAN at the tight evaluation tolerance.

These results illustrate that the efficiency of a neural ODE surrogate depends on both chemical-system complexity and the implementation of the learned right-hand side. Figure 12 summarizes the relationship between inference latency and prediction accuracy across the evaluated chemical systems and numerical configurations.

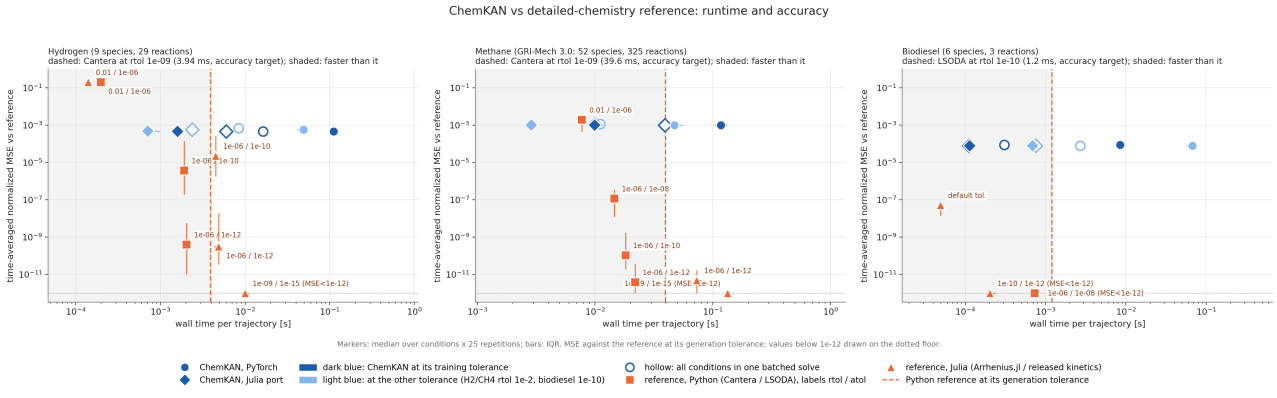


Figure 12: Inference latency and time-averaged normalized trajectory MSE for the three chemical systems, comparing PyTorch and Julia ChemKAN with their numerical reference methods under different solver tolerances. Markers indicate median measurements and error bars the interquartile ranges.

3.6 Interpretability

The learned biodiesel ChemKAN was examined through its univariate edge activation functions and through the behavior of the complete learned kinetic right-hand side. The analysis used the final 156-parameter noise-free

biodiesel checkpoint. No additional training was performed.

The first-layer KAN functions provide a decomposition of how individual normalized state variables contribute to the hidden representation. Figure 13 shows the learned univariate functions connecting the six species concentrations and temperature to the four hidden nodes.

These functions can be inspected individually, but their effects on the predicted reaction rates also depend on the subsequent LeanKAN layer. Therefore, individual edge shapes should not be interpreted directly as chemical reaction laws.

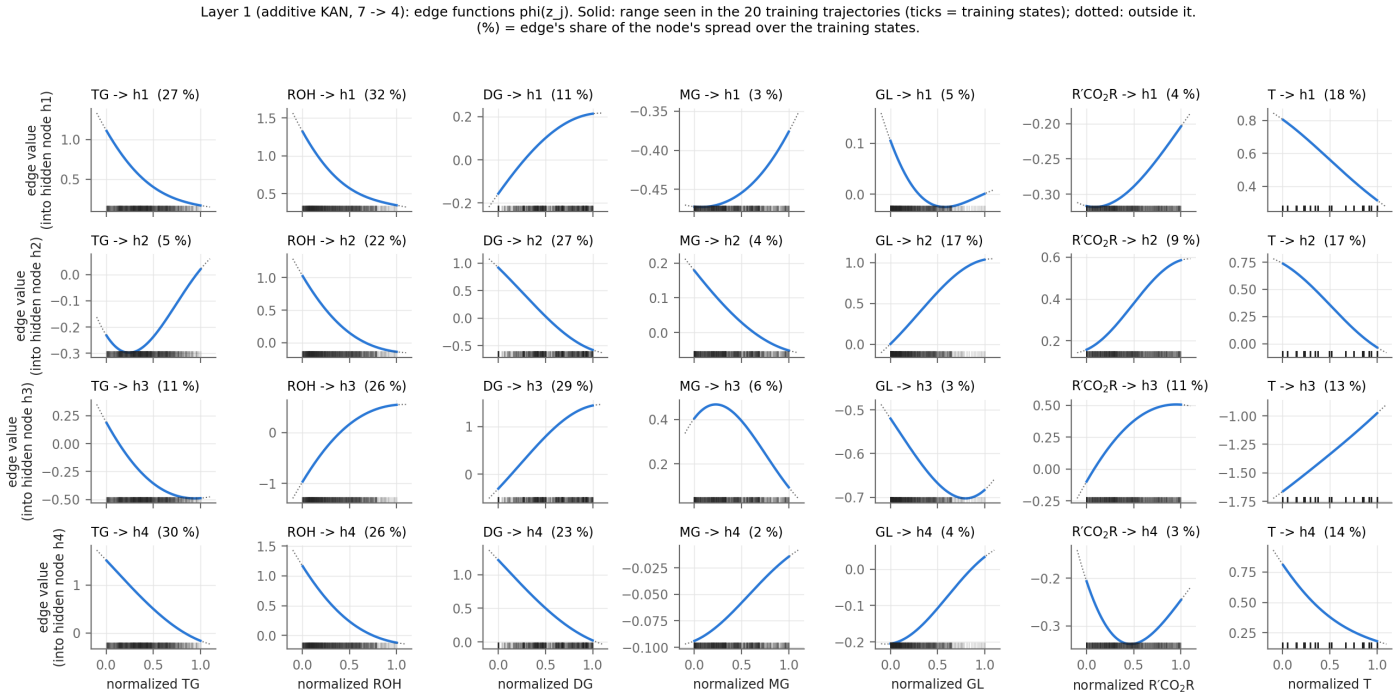


Figure 13: Learned first-layer univariate edge activation functions of the final noise-free biodiesel ChemKAN, evaluated over the observed training-domain input ranges.

For each species, the relative root-mean-square error of the predicted production rate is defined as

$$\mathrm{rRMSE}_k = \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\dot{u}_{k,i}^{\mathrm{pred}} - \dot{u}_{k,i}^{\mathrm{ref}}\right)^2}}{\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\dot{u}_{k,i}^{\mathrm{ref}}\right)^2}}. \tag{31}$$

The metric is evaluated separately on training and held-out reference states.

Figure 14 compares the complete learned ChemKAN kinetic right-hand side with the known Arrhenius kinetics at 600 training and 300 held-out states sampled from clean reference trajectories. The rates are evaluated at the same states, without integrating new trajectories or varying the inputs.

The predicted rates generally follow the reference rates for TG, ROH, DG, GL, and R'CO_2R, with relative RMSE between 0.09 and 0.14 across the training and held-out sets. The largest discrepancy occurs for MG (0.24 training and 0.28 held-out relative RMSE). In particular, ChemKAN predicts positive MG production rates at some states where the reference production rate is approximately zero.

These results indicate that ChemKAN approximates most biodiesel source terms well at the sampled reference states, although discrepancies remain for some species. The learned edge functions allow inspection of the model's functional structure, while the rate comparison provides a quantitative assessment of its agreement with known kinetics. However, this does not establish that the individual learned functions correspond to specific chemical reactions.

Additional read-only interpretability diagnostics were also generated for hydrogen and methane. The detailed discussion here focuses on biodiesel because its known, compact Arrhenius mechanism permits direct validation of the learned kinetic rates. A systematic cross-system interpretability comparison was not undertaken.

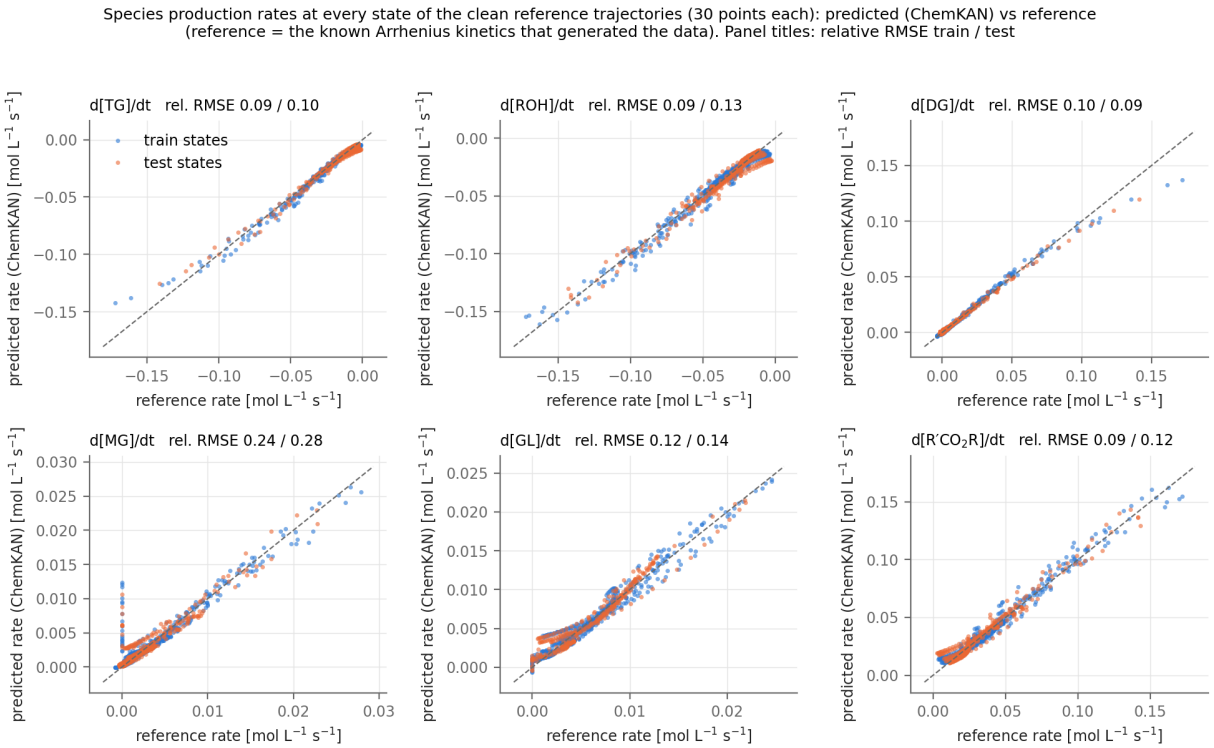


Figure 14: Biodiesel species production rates predicted by the complete learned ChemKAN and the known Arrhenius kinetics at 600 training and 300 held-out reference states. The dashed diagonal represents perfect agreement. Relative RMSE values are shown for each species, separately for training and held-out states.

3.7 Evaluation against proposal success criteria

Table 7: Evaluation against the five success criteria defined in the approved proposal.

Success criterion	**Experimental evidence and observations**
Implementation	An independent PyTorch ChemKAN implementation includes the kinetic core, thermodynamic superstructure, two-stage training, direct autograd, and a validated continuous-FSA backend.
Reproduction	Biodiesel reproduces several reported noise robustness trends under the clarified protocol. The 200k hydrogen model predicts the designated held-out condition with time-averaged MSE 3.507×10^{-4} and has a median absolute ignition-delay error of 1.36% on the Figure 8B conditions. Some quantitative discrepancies remain across initial conditions. The comparison with published hydrogen MSE ranges is sensitive to the loss normalization convention, and agreement is not uniform across the reconstructed initial-condition grid.
Extension	ChemKAN was extended to methane-air oxidation using GRI-Mech 3.0. Stage 2 training reached 200,000 epochs, with the 180k checkpoint selected for evaluation. The held-out time-averaged MSE was 1.163×10^{-3}. The Julia inference port achieved approximately $4.0\times$ acceleration relative to Cantera under the measured configuration.
Comparative analysis	DeepONet was reconstructed and compared with ChemKAN under controlled biodiesel experiments. Published ChemNODE results were used for context. Computational efficiency was measured against detailed numerical chemistry in Python and Julia. See Table 6.
Interpretability	The learned KAN edge functions were visualized and the complete biodiesel kinetic right-hand side was compared with the known Arrhenius rates. The model approximated most species production rates at sampled reference states, although larger discrepancies remained for MG. Individual learned functions were not uniquely identified with specific chemical reactions.

4 Discussion and State-of-the-Art Comparison

4.1 RQ1: Reproducibility of ChemKAN

The independent PyTorch implementation reproduces the central behavior of both original case studies, but exact quantitative reproduction could not be obtained from the paper text alone. In the first paper-only implementation, extensive ablations reduced some discrepancies but the hydrogen thermodynamic stage still failed to reproduce temperature ignition reliably and biodiesel losses remained above the reported levels. The later author clarification and released biodiesel source resolved several settings that materially changed the reproduction [2, 4].

For biodiesel, the revised setup closely reproduces the reported noise-floor behavior and approximately two-fold clean-test degradation at 15% noise, while the reconstructed neural-scaling and DeepONet results remain more sensitive. For hydrogen, the 200k Stage-2 checkpoint reproduces ignition timing and the main trajectory behavior with a dense-grid median time-averaged MSE of 4.91×10^{-4} and 1.36% median absolute ignition-delay error on the Figure 8B conditions. The reproduction is therefore best described claim by claim rather than as a single binary success/failure result.

4.2 RQ2: Transfer to methane-air oxidation

The methane extension demonstrated that the ChemKAN architecture can be applied to a chemical system substantially larger than the hydrogen benchmark. Using 1,892 trainable parameters, the selected model learned the principal ignition and species trajectories of the GRI-Mech 3.0 system.

For the held-out initial condition, the model achieved a time-averaged normalized MSE of 1.163×10^{-3}. Across the 36 investigated initial conditions, the median error was 9.94×10^{-4}. The model reproduced the overall ignition behavior, although discrepancies remained in transient temperature profiles and several intermediate-species trajectories. The broader 441-condition evaluation further demonstrated generalization to unseen initial-condition combinations, with a median time-averaged MSE of 9.21×10^{-4} across the 406 unseen conditions.

Computational efficiency depended strongly on the execution environment. The Julia inference port was approximately four times faster than Cantera and 13.5 times faster than Arrhenius.jl under the measured configurations. The PyTorch implementation was slower than Cantera for individual trajectories.

The extension therefore provides evidence of transferability to a larger chemical system, while also demonstrating that training stability, physical fidelity, and practical acceleration must be evaluated separately.

4.3 RQ3: Comparison with neural and numerical references

DeepONet predicts the future state directly from the initial condition and time, avoiding ODE integration during inference [11]. The original ChemKAN study used DeepONet as the neural baseline for biodiesel production and reported stronger overfitting under noisy training observations [8]. In this project, the 308-parameter DeepONet architecture was reconstructed using the published model size and clarifications from the authors. Because the exact activation functions and optimization settings were not fully specified, the selected tanh reconstruction is treated as a behavior-matching implementation rather than an exact recovery of the authors' original DeepONet.

The results in Figure 4 and Table 3 show that the selected 308-parameter DeepONet achieved comparable or lower clean-test MSE than the 156-parameter ChemKAN at several lower observation-noise levels. At 10% and 15% noise, however, ChemKAN achieved approximately 2.7-fold lower clean-test MSE. The single-seed reproduction therefore supports a relative advantage for ChemKAN at higher noise levels, but not uniform superiority across all investigated noise levels.

ChemNODE provides a published neural-ODE baseline for hydrogen-air combustion. Like ChemKAN, it learns chemical source terms that are integrated to obtain thermochemical trajectories. However, ChemNODE uses separately trained neural networks for individual thermochemical variables, whereas ChemKAN combines the kinetic and thermodynamic components within a single chemistry-specific KAN architecture. ChemNODE modeled six species and temperature, while the hydrogen ChemKAN models represent nine species and temperature [12, 8].

ChemNODE achieved approximately $2.3\times$ acceleration over the detailed hydrogen-air mechanism for integration to steady state [12]. The original ChemKAN paper reported approximately $2.0\times$ acceleration using

Table 8: Architectural and computational comparison of ChemNODE and hydrogen ChemKAN. Published speed-ups and measurements from this work use different hardware, reference solvers, and timing protocols and are not directly comparable across studies.

Model	Nets	Params	States	Reference	Speed-up
ChemNODE (published)	7	637[a]	6 species + T	Detailed chemistry	$2.3\times$
ChemKAN (published)	1	344	9 species + T	Arrhenius.jl	$2.0\times$
ChemKAN (PyTorch)	1	344	9 species + T	Cantera	$0.035\times$
ChemKAN (Julia)	1	344	9 species + T	Cantera	$2.5\times$
ChemKAN (Julia)	1	344	9 species + T	Arrhenius.jl	$6.3\times$

[a] ChemNODE parameter count estimated in [8] using standard MLP parameterizations (seven networks with 91 parameters each).

Arrhenius.jl as the detailed-chemistry reference [8]. These published results are included for architectural and methodological context; ChemNODE was not independently retrained or benchmarked in this project.

The primary ChemKAN reproduction was implemented and trained in PyTorch. Under the local hydrogen inference benchmark, the PyTorch implementation required approximately 111.5 ms per trajectory, compared with 3.94 ms for Cantera. Thus, despite reproducing the principal combustion behavior, the direct PyTorch implementation did not achieve the computational acceleration reported in the original ChemKAN study.

To investigate the influence of the execution environment, an inference-only implementation was subsequently developed in Julia using the parameters learned in PyTorch. No additional model training or parameter fitting was performed. Agreement between the PyTorch and Julia predictions was verified before benchmarking. The Julia port required approximately 1.60 ms per hydrogen trajectory, corresponding to $2.5\times$ acceleration over Cantera and $6.3\times$ over Arrhenius.jl under the evaluated configurations.

The local benchmarks therefore show that inference efficiency depends substantially on the numerical implementation, even when the learned parameters remain unchanged. Nevertheless, the published ChemNODE and ChemKAN speed-ups cannot be interpreted as a direct performance ranking against the present results, because the numerical references, hardware, and timing tasks differ.

The effect of mechanism complexity was also evident across the investigated systems. For the three-reaction biodiesel system, direct Julia integration of the known kinetics remained faster than Julia ChemKAN. In contrast, the Julia ChemKAN ports achieved acceleration over detailed chemistry for hydrogen and methane, with larger measured speed-ups for methane. These findings suggest that potential computational savings depend on both chemical-system complexity and the execution environment, rather than on neural-network parameter count alone.

4.4 Reproducibility lessons

The project shows that scientific machine-learning reproduction requires source-level provenance in addition to architectural similarity. Several examples materially affected the experiments: the printed biodiesel reaction ordering differs from the released code; printed Eq. (18) sums over time while the released Julia example uses time-averaged `Flux.mse`; the paper describes a Swish base activation while the authors clarified that the final models did not use it; the paper text omits $\phi = 0.5$ from the hydrogen list although the 36-condition design includes it; and the released biodiesel example uses `AutoTsit5(Rosenbrock23)` rather than only Tsit5. The hydrogen clarification additionally supplied the 100-point grid, dense Stage-1 temperature interpolation, 10^{-5} thermodynamic initialization, and the 10^5–2×10^5 training budget.

Where later source material or author clarification resolves a discrepancy, the final reproduction adopts the directly supported setting and states its provenance instead of treating it as a convention introduced by this project. Earlier paper-only results are retained because they show which details were necessary to obtain the later reproduction.

The FSA comparison provides a second numerical lesson. Matching architecture, data, initialization, optimizer, and nominal tolerances does not by itself guarantee an identical optimization trajectory when the sensitivity method changes the variables included in the adaptive solve. The close PyTorch-FSA/Julia training trajectory is therefore reported as evidence about numerical implementation sensitivity, while the proposed error-control explanation remains explicitly a hypothesis.

The cross-language inference comparison also shows that numerical and software implementation choices can dominate the measured computational efficiency of a small learned surrogate.

4.5 Limitations

- Most expensive final comparisons use one random seed, so reported ratios are single-run measurements rather than uncertainty estimates. For methane, checkpoint selection was based on training-condition accuracy, with the 180k checkpoint outperforming the final 200k checkpoint. These results do not establish convergence or consistency across different random initializations.
- The released biodiesel example differs from the printed paper in several implementation details; every adopted source-code setting is attributed to the released source rather than to the paper.
- Which biodiesel reaction ordering generated the published Figure 3 trajectories was not explicitly confirmed by the authors; controlled trajectory evidence points to the released-code ordering.
- Hydrogen reference trajectories are generated with Cantera rather than the authors' Arrhenius.jl CVODE_BDF workflow, although the authors reported cross-checking their data against Cantera.
- The 441-condition Figure 8A grid is reconstructed from the figure/count because the exact grid is not tabulated in the paper.
- The exact hydrogen/methane normalized-time implementation with t_{ref} equal to the full trajectory horizon is documented from the PyTorch reproduction itself; no released hydrogen implementation is available for source-level confirmation.
- Learned KAN edge functions are inspectable but are not automatically identifiable with individual chemical reactions.
- Agreement between predicted and reference reaction rates at sampled states does not imply identification of the underlying chemical mechanism. In particular, ChemKAN predicts nonzero MG production at some states where the known kinetics predict approximately zero.
- Methane generalization was evaluated using one designated held-out test condition and a broader 21×21 grid containing 35 training and 406 unseen initial conditions. The unseen conditions include the designated held-out case and remain within the investigated temperature and equivalence-ratio ranges. Therefore, these results assess generalization to unseen initial-condition combinations but do not establish extrapolation beyond those ranges.
- The combustion models use a soft element-conservation penalty rather than an exact conservation constraint. Agreement with the reference trajectories does not guarantee physical conservation at every time point.
- Runtime comparisons were measured on one Apple M1 machine and depend on the implementation language, batching, solver tolerances, and numerical precision. The local Cantera and Arrhenius.jl comparisons use different numerical configurations, so the measured speed-ups are implementation-specific and should not be interpreted as universal acceleration factors or direct reproductions of the published speed-ups.

5 Conclusion

This project developed an independent PyTorch implementation of ChemKAN and investigated its reproducibility on biodiesel production and hydrogen-air combustion. The principal model behavior was recovered, although closer quantitative reproduction required methodological clarifications from the authors and information from their released biodiesel implementation.

For biodiesel, the experiments reproduced several reported noise-robustness trends. For hydrogen, the 200k Stage-2 checkpoint achieved a held-out time-averaged normalized MSE of 3.507×10^{-4} and a median absolute ignition-delay error of 1.36% on the Figure 8B conditions. Predictive accuracy nevertheless varied across initial conditions. Continuous forward sensitivity analysis was also implemented and validated, with controlled biodiesel experiments showing close agreement with the released Julia training behavior.

The methane extension demonstrated the applicability of ChemKAN to a larger chemical system. The selected 180k Stage-2 checkpoint achieved a held-out time-averaged normalized MSE of 1.163×10^{-3} and generalized across the investigated initial-condition domain, although training remained non-monotonic and some species and temperature discrepancies persisted.

The comparative experiments highlighted the importance of both training conditions and numerical im-

plementation. In biodiesel, ChemKAN achieved lower clean-test error than the selected tanh DeepONet at higher observation-noise levels, but not uniformly across all noise levels. In combustion, Julia inference ports of the PyTorch-trained ChemKAN models achieved approximately $2.5\times$ and $4.0\times$ speed-ups over Cantera for hydrogen and methane, respectively, whereas direct PyTorch inference was slower under the measured configurations.

Finally, the biodiesel interpretability analysis demonstrated the inspectability of learned KAN functions and identified discrepancies in some instantaneous species production rates despite agreement with reference trajectories. Overall, the results emphasize that reproducibility, generalization, physical accuracy, and computational efficiency must be evaluated separately when assessing learned chemical-kinetics surrogates.

6 Domain-Specific Lecture

Selected lecture: 165.171 AI in Chemistry.
Completed (certificate issued): 24.07.2026

References

[1] Pete Chapman et al. *CRISP-DM 1.0: Step-by-step data mining guide*. Tech. rep. SPSS Inc., 2000. URL: https://public.dhe.ibm.com/software/analytics/spss/documentation/modeler/14.2/es/CRISP-DM.pdf.

[2] ChemKAN authors. *Methodological clarification by personal communication*. Received 22 September 2026. Sept. 2026.

[3] Ricky T. Q. Chen. *torchdiffeq*. Commit: 657943acefa826ef04c025ebeb1ff5e9d60dc268, 2018. URL: https://github.com/rtqichen/torchdiffeq.

[4] DENG-MIT. *ChemKAN: released Julia biodiesel example*. Biodiesel example audited at commit d7aa5abe. 2026. URL: https://github.com/DENG-MIT/ChemKAN.

[5] David G. Goodwin, Harry K. Moffat, Ingmar Schoegl, Raymond L. Speth, and Bryan W. Weber. *Cantera: An Object-oriented Software Toolkit for Chemical Kinetics, Thermodynamics, and Transport Processes*. Version 3.0.0. URL: https://www.cantera.org.

[6] Weiqi Ji et al. *Arrhenius.jl: A Differentiable Combustion Simulation Package*. 2021. arXiv: 2107.06172 [physics.chem-ph]. URL: https://arxiv.org/abs/2107.06172.

[7] Benjamin C. Koenig, Suyong Kim, and Sili Deng. *KAN-ODEs: Kolmogorov-Arnold Network Ordinary Differential Equations for Learning Dynamical Systems and Hidden Physics*. 2024. URL: https://arxiv.org/abs/2407.04192.

[8] Benjamin C. Koenig, Suyong Kim, and Sili Deng. "ChemKANs for combustion chemistry modeling and acceleration". In: *Physical Chemistry Chemical Physics* 27 (2025), pp. 17313–17330. URL: https://doi.org/10.1039/D5CP02009C.

[9] Benjamin C. Koenig, Suyong Kim, and Sili Deng. "LeanKAN: a parameter-lean Kolmogorov-Arnold network layer with improved memory efficiency and convergence behavior". In: *Neural Networks* 192 (2025), p. 107883. URL: https://doi.org/10.1016/j.neunet.2025.107883.

[10] Ziming Liu et al. "KAN: Kolmogorov–Arnold Networks". In: *International Conference on Learning Representations*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu. Vol. 2025. 2025, pp. 70367–70413. URL: https://proceedings.iclr.cc/paper_files/paper/2025/file/afaed89642ea100935e39d39a4da602c-Paper-Conference.pdf.

[11] Lu Lu, Pengzhan Jin, and George Em Karniadakis. *DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators*. 2019. URL: https://arxiv.org/abs/1910.03193.

[12] Opeoluwa Owoyele and Pinaki Pal. "ChemNODE: A neural ordinary differential equations framework for efficient chemical kinetic solvers". In: *Energy and AI* 7 (2022), p. 100118. URL: https://doi.org/10.1016/j.egyai.2021.100118.

[13] Christopher Rackauckas and Qing Nie. “DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia”. In: *Journal of Open Research Software* (May 2017).

[14] SciML. *Mathematics of Sensitivity Analysis: Forward Sensitivity Analysis*. 2026. URL: https://docs.sciml.ai/SciMLSensitivity/dev/sensitivity_math/.

[15] Gregory P. Smith, David M. Golden, Michael Frenklach, Nigel W. Moriarty, Boris Eiteneer, et al. *GRI-Mech 3.0*. 1999. URL: http://combustion.berkeley.edu/gri-mech/.

[16] Pauli Virtanen et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature methods* 17.3 (2020), pp. 261–272.

A Implementation and Training Settings

Table 9: Main ChemKAN architecture, training, and evaluation settings. Solver tolerances are listed as relative/absolute tolerances.

	Biodiesel	**Hydrogen**	**Methane extension**
State	6 concentrations; T input only	9 species + T (Stage 2)	52 species + T (Stage 2)
Train/test ICs	20 / 10	35 / 1	35 / 1
Trajectory	30 s, 30 points	0.6 ms, 100 points	5 ms, 100 points
Core width	4	3	3
RBF basis count	3	5	5
n_{mu}	2	3	3
Base activation	off	off	off
Parameters	156	344	1,892
Optimizer	Adam	Adam	Adam
Learning rate	10^{-2} in author-match	2×10^{-3}	2×10^{-3}
Training solver	Tsit5, $10^{-2}/10^{-6}$	Tsit5, $10^{-6}/10^{-8}$	Tsit5, $10^{-6}/10^{-8}$
Final evaluation	Tsit5, $10^{-10}/10^{-12}$	float64 Tsit5, $10^{-10}/10^{-12}$	Selected Stage-2 180k checkpoint; Tsit5, $10^{-6}/10^{-8}$
Sensitivity backend	FSA	Direct autograd	Direct autograd
Training budget	10k (main noise runs)	100k Stage 1 + 200k Stage 2	Stage 1 trained to 54.8k; archived 50k checkpoint used + 200k Stage 2
Selected evaluation checkpoint	Final 10k checkpoint	Stage 2 epoch 200,000	Stage 2 epoch 180,000

B Source and Reproducibility Notes

A report-specific claim-to-evidence map is supplied with the report source as `REPORT_CLAIM_EVIDENCE_MAP.md`. It identifies the literature, author-clarification, implementation, checkpoint and numerical-artifact basis for major methodological and empirical claims, and marks evidence unavailable from the public release.

Project repository: https://github.com/bnkglu/chemkan-tuwien/

The repository contains the PyTorch implementation, experiment configurations, saved checkpoints, evaluation scripts, results, and dependency information.

C Training Histories

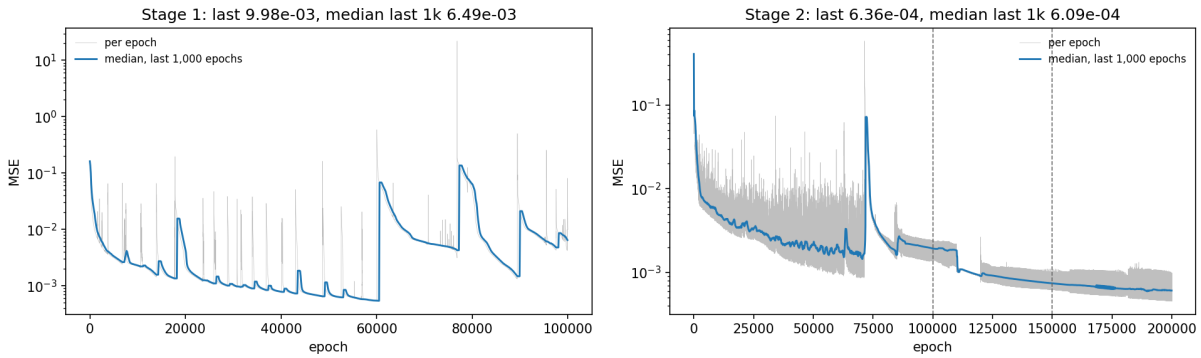


Figure 15: Hydrogen ChemKAN training histories for Stage 1 and Stage 2. Stage 1 was trained for 100,000 epochs, followed by 200,000 Stage-2 epochs. The stages have different prediction targets and the loss magnitudes should not be interpreted as directly comparable measures of difficulty.

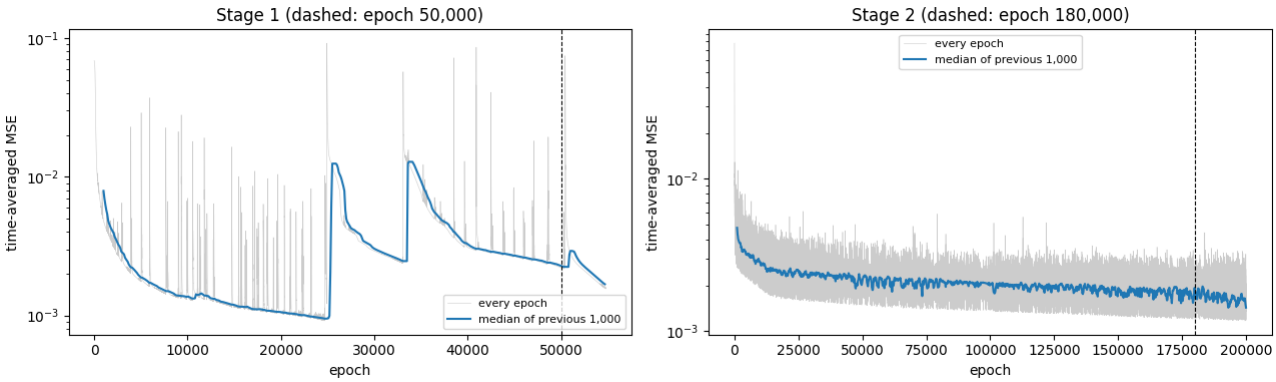


Figure 16: Methane Stage-1 and Stage-2 training histories (seed 0). Grey curves show the logged time-averaged MSE at each epoch, while blue curves show the median over the preceding 1,000 epochs. Dashed vertical lines indicate the archived Stage-1 checkpoint used to initialize Stage 2 (50k) and Stage-2 evaluation checkpoint (180k). Stage 2 training continued to 200k. The two stages have different prediction targets and their loss magnitudes are not directly comparable.

D Additional Interpretability Plots

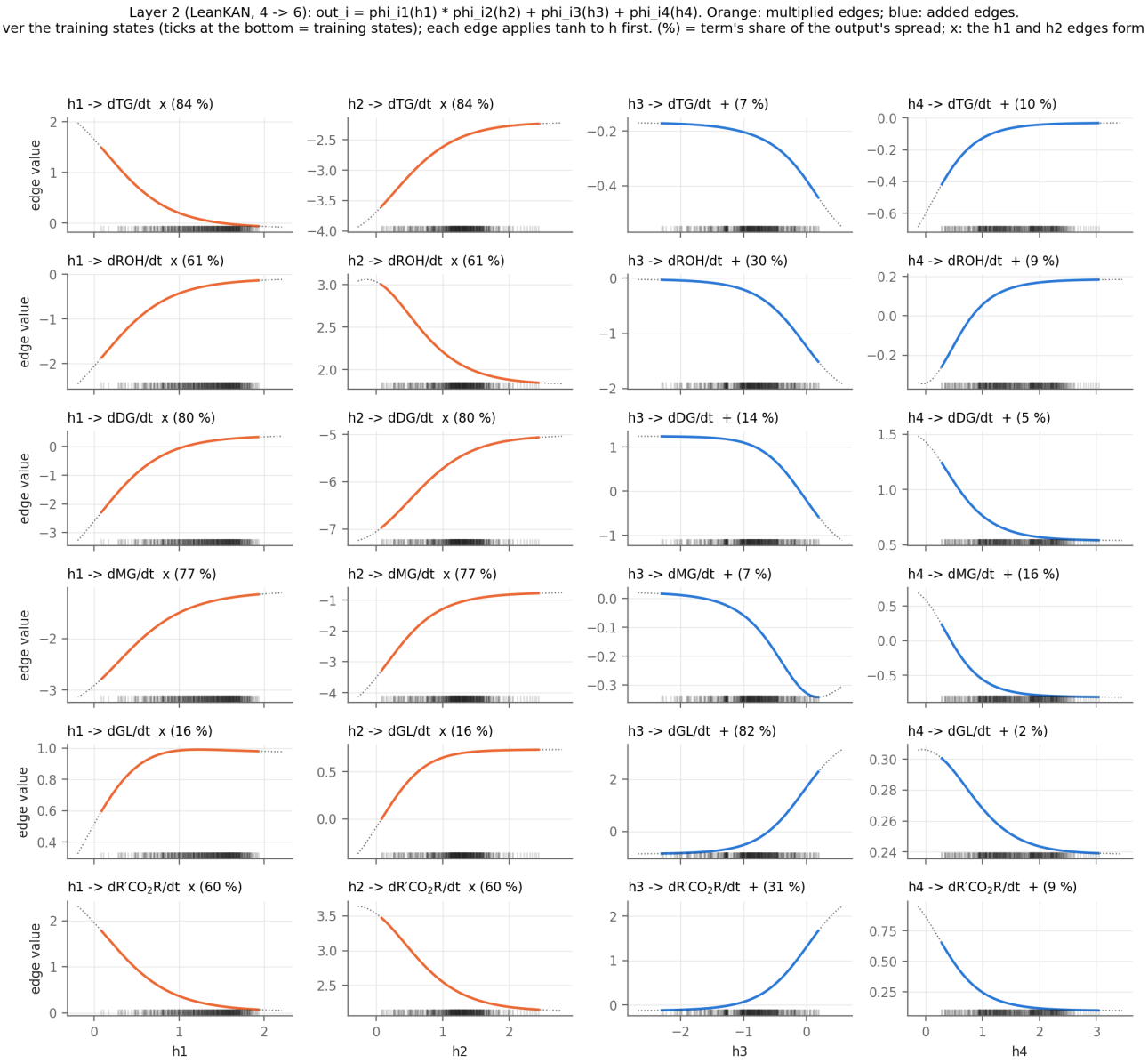


Figure 17: Learned second-layer LeanKAN functions in the biodiesel kinetic model. Orange curves represent the two factors forming each multiplicative contribution, while blue curves represent additive contributions. The displayed percentages indicate each output term's standard deviation as a proportion of the sum of term standard deviations over the training states. Both orange factors share the percentage of their combined product term. These values do not imply causal feature importance.